

Maestría en Inteligencia Artificial

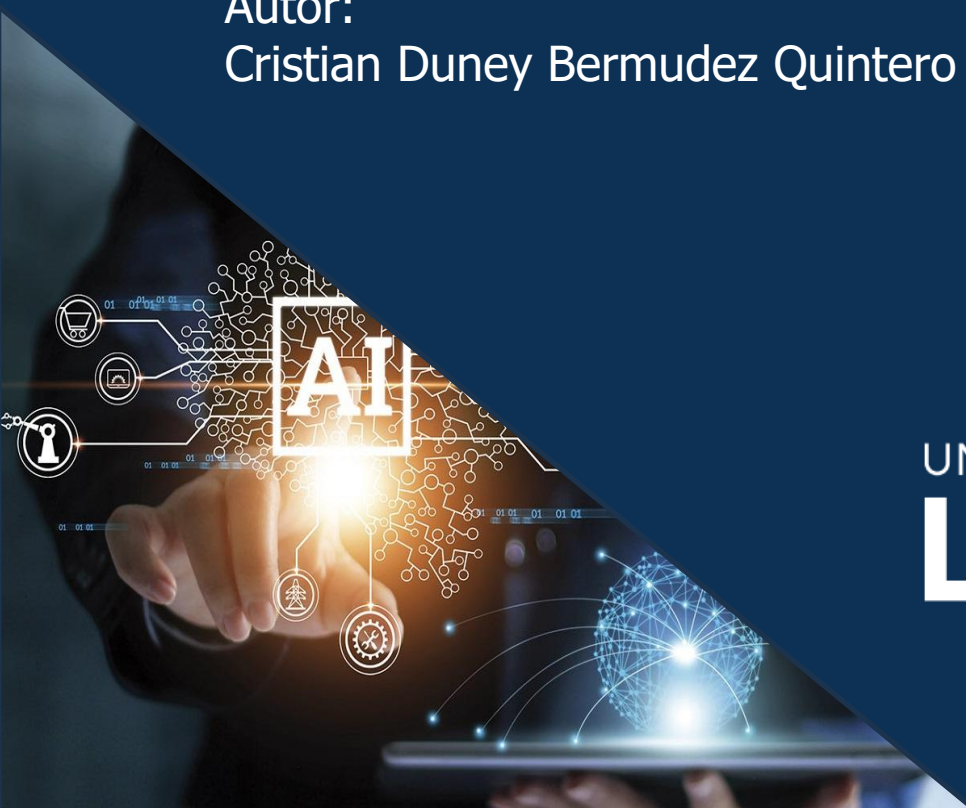
Aplicaciones I

Unidad 2:

Aplicaciones web analíticas con Python y
Dash
2025

Autor:

Cristian Duney Bermudez Quintero



UNIVERSIDAD DE
LASALLE

Índice

Capítulo 1	3
Fundamentos de Python: Lenguaje, Herramientas y Ecosistema	3
1.1. Python.....	4
1.2. Implementaciones de Python	5
1.3. Bibliotecas de Python	7
1.4. Framework en Python.....	9
1.5. IDEs tradicionales (desarrollo de software)	10
1.6. Entornos interactivos (análisis de datos y experimentación)	13
1.7. Sintaxis y programación básica en Python.....	15
Capítulo 2	24
Organización del Código y Herramientas de Desarrollo en Python	24
2.1. Organización del código en scripts y módulos.....	26
2.2. Convenciones de estilo en Python (PEP 8)	28
2.3. Uso de entornos virtuales para la gestión de dependencias.....	31
2.3.1. Pasos para crear un entorno virtual	32
2.4. Manejo de archivos y bases de datos	34
2.4.1. Lectura y escritura de archivos CSV, JSON y Excel	34
2.4.2. Conexión a bases de datos SQL y NoSQL.....	36
2.5. Creación de interfaces graficas de usuarios (GUI) en Python	37
Capítulo 3	43
Desarrollo y despliegue de aplicaciones web analíticas con Python	43
3.1. Python y su Impacto en el Desarrollo Web	44
3.2. Visualización de datos en Dash	45
3.2.1. Plotly en Dash (la opción nativa y recomendada)	46
3.3. Desarrollo de aplicaciones web analíticas con Python y Dash	50
3.4. Manejo de datos con Dash	53
3.5. Autenticación y seguridad en Dash	55
3.6. Despliegue de aplicaciones web dinámicas con Dash en la nube	57
Referencias	63

Capítulo 1

Fundamentos de Python: Lenguaje, Herramientas y Ecosistema

Python se ha consolidado como uno de los lenguajes de programación más influyentes en la actualidad gracias a su sencillez, flexibilidad y gran variedad de aplicaciones. Este lenguaje no solo resulta accesible para quienes inician en la programación, sino que también es lo suficientemente robusto para proyectos complejos en campos como el desarrollo web, la ciencia de datos, la inteligencia artificial o la automatización de procesos. Su crecimiento sostenido se explica por una sintaxis intuitiva, el respaldo de una comunidad activa y la disponibilidad de un extenso conjunto de bibliotecas y frameworks que amplían sus posibilidades.

A lo largo de este capítulo se presentarán los fundamentos esenciales de Python. Se abordarán sus características más relevantes, las bibliotecas de mayor uso en análisis de datos y aprendizaje automático, y los frameworks que permiten construir aplicaciones web de manera eficiente. También se explicarán los entornos de desarrollo más utilizados, tanto integrados (IDEs) como interactivos, los cuales facilitan el aprendizaje y optimizan la práctica de la programación. Finalmente, se revisarán aspectos básicos de la sintaxis y de la programación estructurada en Python, con el propósito de ofrecer una base sólida que permita aprovechar de forma efectiva el potencial de este lenguaje en distintos proyectos.



1.1. Python

Python se ha consolidado como uno de los lenguajes de programación más utilizados en la actualidad, con aplicaciones que abarcan desde el desarrollo de aplicaciones web y la creación de software hasta el análisis y la ciencia de datos, así como el aprendizaje automático (Machine Learning, ML). Su amplia adopción se explica por la eficiencia y simplicidad que ofrece, características que lo convierten en una herramienta accesible para quienes inician en la programación y, al mismo tiempo, potente para profesionales con experiencia.

con diferentes sistemas operativos y plataformas, lo que permite implementarlo en entornos muy diversos. Cabe resaltar que Python es un software de uso libre y gratuito, disponible para descarga en su sitio oficial: <https://www.python.org/downloads/>



Ventajas de Python

Python ofrece varios beneficios, entre los cuales se destacan:

- ☒ Su estructura, cercana al lenguaje natural, facilita la comprensión del código y reduce la complejidad al momento de programar.
- ☒ Permite resolver problemas con menos líneas de código en comparación con otros lenguajes, lo que agiliza la escritura y mantenimiento de programas.
- ☒ Incluye un conjunto variado de módulos y funciones ya desarrollados, lo que evita que los programadores deban partir desde cero en cada proyecto.
- ☒ Se integra sin dificultad con tecnologías como Java, C o C++, favoreciendo la comunicación entre distintos sistemas y aplicaciones.
- ☒ Comunidad activa y global que participan en foros, grupos y proyectos abiertos, generando un entorno de apoyo constante.
- ☒ Existen manuales, tutoriales, cursos y documentación oficial disponibles en línea, lo que facilita su estudio y dominio progresivo.
- ☒ Puede ejecutarse en sistemas operativos como Windows, macOS, Linux o Unix, lo que asegura versatilidad y portabilidad en diferentes entornos de trabajo.

Sabías que ...

Python fue desarrollado en 1991 por Guido van Rossum, quien eligió su nombre en referencia al grupo de comedia británico Monty Python y no a la serpiente. Desde sus inicios, el lenguaje se diseñó con el propósito de ser claro y fácil de leer, lo que contribuyó a su rápida popularidad. Gracias a la gran cantidad de bibliotecas disponibles, se ha convertido en una herramienta ampliamente utilizada en ámbitos como el desarrollo web, la ciencia de datos y la inteligencia artificial.



1.2. Implementaciones de Python

Python se caracteriza por su gran versatilidad, lo que ha permitido su aplicación en múltiples áreas del conocimiento y la industria. Su combinación de simplicidad y potencia lo convierte en una herramienta fundamental para el diseño de soluciones eficientes, tanto en el ámbito del desarrollo tecnológico como en procesos de automatización y análisis de información. A continuación, se presentan algunos de los principales escenarios en los que este lenguaje es empleado.

Desarrollo web del lado del servidor

El desarrollo web en el backend se centra en las funciones internas que permiten a los sitios interactuar con los usuarios, gestionar bases de datos, comunicarse con otros servicios y garantizar la seguridad en la transmisión de información.

Python resulta especialmente adecuado para este tipo de tareas, ya que dispone de un amplio conjunto de bibliotecas y frameworks que simplifican la implementación de funcionalidades avanzadas. Gracias a estos marcos de trabajo, es posible crear la estructura básica de una aplicación web en poco tiempo, realizar pruebas de manera ágil y desarrollar proyectos sin depender de herramientas adicionales.

Automatización mediante scripts en Python

Los lenguajes de scripting se utilizan para reducir o eliminar tareas repetitivas que normalmente requieren de una persona. Python destaca en este campo por su facilidad de uso y su sintaxis clara, lo que lo convierte en una opción muy popular para la creación de scripts simplificando actividades como:

- Renombrar múltiples archivos a la vez
- Convertir un tipo de archivo en otro
- Eliminar duplicados de un archivo de texto

- Realizar operaciones matemáticas simples
- Enviar correos electrónicos
- Descargar contenido de la web
- Realizar análisis básicos de registros
- Buscar errores en archivos

Ciencia de Datos, Analítica de Datos y Machine Learning

La ciencia de datos se centra en extraer información valiosa de grandes volúmenes de datos, mientras que la analítica de datos se enfoca en analizar, interpretar y presentar esos datos para facilitar la toma de decisiones informadas. El ML, por su parte, enseña a las máquinas de la capacidad de reconocer patrones en los datos y mejorar su desempeño sin necesidad de ser programadas de manera explícita. Este enfoque permite generar modelos capaces de realizar predicciones con un alto nivel de precisión. En este campo, Python se ha convertido en una de las herramientas preferidas por los científicos de datos, ya que ofrece un ecosistema robusto de bibliotecas y recursos que facilitan tareas como:

- Limpiar y corregir datos incorrectos
- Extraer y seleccionar características relevantes de los datos
- Etiquetar datos con nombres significativos
- Obtener estadísticas detalladas de los datos
- Visualizar los datos mediante gráficos como barras, líneas, circulares y histogramas

Además, los científicos de datos emplean las bibliotecas de machine learning de Python para entrenar modelos y desarrollar clasificadores que organizan los datos con alta precisión, como en tareas de clasificación de imágenes, texto, tráfico de red, reconocimiento de voz y facial. Python también es utilizado en el aprendizaje profundo, una técnica avanzada de machine learning que permite resolver problemas complejos a través de redes neuronales.

Desarrollo de software

Python es una herramienta popular entre los desarrolladores de software, quienes lo emplean en diversas actividades como:

- Hacer seguimiento de errores en el código
- Automatizar la creación de software
- Gestionar proyectos de software
- Desarrollar prototipos de software
- Crear aplicaciones de escritorio utilizando bibliotecas de interfaz gráfica
- Desarrollar juegos, desde los simples hasta los más complejos

Automatización de pruebas de software

La prueba de software se centra en verificar que los resultados reales coincidan con los esperados para garantizar que el software funcione correctamente. Para esta tarea, en Python se utilizan diferentes frameworks de pruebas unitarias, entre los que destacan Unittest, Robot y PyUnit, los cuales permiten evaluar de manera estructurada las funciones implementadas.

Además, las pruebas se automatizan utilizando herramientas de integración y despliegue continuo (CI/CD), como Travis CI y Jenkins, que ejecutan los scripts de prueba de Python de forma automática y proporcionan los resultados de las pruebas cada vez que se hacen cambios en el código.



Características principales de Python

Este lenguaje de programación presenta una serie de propiedades que explican su popularidad:

☒ **Lenguaje interpretado**

Python ejecuta las instrucciones de manera secuencial, línea por línea. En caso de encontrar un error, la ejecución se interrumpe en el punto exacto donde se produce, lo que facilita la localización y corrección del problema.

☒ **Facilidad de uso**

Su sintaxis fue diseñada para ser clara y legible, utilizando palabras clave cercanas al inglés. A diferencia de otros lenguajes, no emplea llaves para delimitar bloques de código.

☒ **Tipado dinámico**

No es necesario declarar de forma explícita el tipo de las variables, ya que este se determina en el momento de la ejecución. Esto agiliza el proceso de programación y aporta flexibilidad en el desarrollo de aplicaciones.

☒ **Lenguaje de alto nivel**

Python permite concentrarse en la lógica de los programas sin preocuparse por aspectos técnicos de bajo nivel, como la gestión de memoria o la arquitectura interna del sistema.

☒ **Programación orientada a objetos**

En este lenguaje, prácticamente todo se maneja como objetos, aunque también admite otros paradigmas, como la programación estructurada y la programación funcional.

1.3. Bibliotecas de Python

En el contexto de la programación, una biblioteca se entiende como un conjunto de módulos y funciones ya desarrollados que pueden incorporarse a un proyecto para simplificar el trabajo del

programador y evitar la escritura de código desde cero. Python ofrece de manera predeterminada una biblioteca estándar, la cual integra herramientas útiles para distintos tipos de aplicaciones. A esto se suma la existencia de más de 137.000 bibliotecas externas, creadas por la comunidad, que abarcan ámbitos tan variados como el desarrollo web, la ciencia de datos, la automatización de procesos y el aprendizaje automático.

Figura 1

Bibliotecas populares de Python

Biblioteca	Propósito Principal	Ámbito de Aplicación	Características Clave
NumPy	Operaciones numéricas y manipulación de arreglos	Cálculos científicos, álgebra lineal	Manejo eficiente de arrays multidimensionales, integración con C/C++.
Pandas	Manipulación de datos tabulares y series temporales	Ciencia de datos, análisis de datos, ML	Estructuras de datos flexibles (<code>DataFrame</code> , <code>Series</code>), procesamiento eficiente.
Matplotlib	Visualización estática de datos en gráficos 2D y 3D	Ciencia de datos, análisis estadístico	Gráficos de barras, líneas, dispersión, 3D, alta personalización.
Seaborn	Visualización estadística con gráficos atractivos	Ciencia de datos, análisis estadístico	Basado en Matplotlib, gráficos avanzados como boxplots y heatmaps.
Plotly	Visualización interactiva de datos	Ciencia de datos, análisis estadístico, dashboards	Gráficos interactivos en 2D y 3D, visualización de datos en tiempo real.
Requests	Realización de solicitudes HTTP	Desarrollo web, integración de API	Envío de solicitudes HTTP, manejo de respuestas y parámetros.
OpenCV-Python	Procesamiento de imágenes y visión artificial	Visión por computadora, procesamiento de imágenes	Lectura y escritura de imágenes, análisis de video, modelado en 3D.
Keras	Redes neuronales profundas	Machine learning, inteligencia artificial	Framework de aprendizaje profundo, integración con TensorFlow.
scikit-learn	Machine learning y minería de datos	Machine learning, análisis predictivo	Algoritmos de clasificación, regresión, clustering y reducción de dimensionalidad.
TensorFlow	Desarrollo de modelos de machine learning y deep learning	ML, inteligencia artificial, aprendizaje profundo	Redes neuronales avanzadas, modelos supervisados y no supervisados, optimización.

Nota. OpenAI. (2025, 15 de Marzo). [Imagen generada por IA con ChatGPT: Bibliotecas populares de Python]. ChatGPT. <https://chat.openai.com/>

Sabías que ...

Python cuenta con un repositorio oficial denominado PyPI (Python Package Index), en el que se alojan más de 400,000 de paquetes disponibles para su uso. La amplia comunidad de desarrolladores ha permitido crear bibliotecas para prácticamente de cualquier tipo de tarea, que van desde aplicaciones de inteligencia artificial hasta el diseño de videojuegos.



1.4. Framework en Python

Un framework o marco en Python puede entenderse como un conjunto estructurado de módulos y paquetes cuyo propósito es simplificar el desarrollo de aplicaciones. Los módulos agrupan fragmentos de código relacionados, mientras que los paquetes reúnen varios de estos módulos bajo una misma organización. Gracias a esta arquitectura, los frameworks permiten a los desarrolladores trabajar de manera más eficiente, ya que se encargan de abstraer detalles técnicos como la gestión de la comunicación en aplicaciones web o la optimización del rendimiento de los programas.

En términos generales, los frameworks en Python se dividen en dos categorías principales:

- **Frameworks de pila completa:** proporcionan casi todas las herramientas necesarias para construir aplicaciones web de gran complejidad y escala.
- **Micromarcos:** ofrecen únicamente las funcionalidades esenciales para proyectos más sencillos, aunque permiten extender sus capacidades mediante complementos y extensiones cuando es necesario.

Principales framework de Python

Existen diversos marcos que han ganado popularidad por optimizar el proceso de desarrollo en Python. Entre los más destacados se encuentran:

- **Django:** Considerado uno de los frameworks de pila completa más robustos y utilizados para el desarrollo web a gran escala. Incluye un servidor integrado para pruebas, un motor de plantillas para la parte visual, y múltiples herramientas de seguridad que facilitan la construcción de aplicaciones confiables.
- **Flask:** Se clasifica como un micromarco, ideal para aplicaciones más ligeras. Destaca por su amplia comunidad, documentación clara y funcionalidades básicas como servidor de pruebas, motor de plantillas y soporte para pruebas unitarias. Además, admite extensiones que incorporan validación de datos, gestión de bases de datos y mecanismos de seguridad.

- **TurboGears:** Diseñado para agilizar el desarrollo de aplicaciones web, combina una estructura de base de datos, herramientas de administración de proyectos, motores de plantillas y sistemas de seguridad que facilitan la creación de aplicaciones completas en menos tiempo.
- **Apache MXNet:** Orientado al aprendizaje profundo, este framework se caracteriza por su rapidez y flexibilidad. Es ampliamente utilizado en investigación y desarrollo de aplicaciones de inteligencia artificial. Admite varios lenguajes de programación, como Java, C++, R y Perl, y ofrece un ecosistema sólido para construir modelos de visión por computadora y procesamiento de lenguaje natural.
- **PyTorch:** Derivado de la biblioteca Torch, es uno de los marcos más reconocidos en el ámbito del aprendizaje automático. Se aplica en áreas como la robótica, la visión por computadora y el procesamiento de lenguaje natural. Una de sus mayores ventajas es la posibilidad de ejecutar modelos tanto en CPU como en GPU, lo que facilita el trabajo con grandes volúmenes de datos, incluyendo imágenes y videos.

1.5. IDEs tradicionales (desarrollo de software)

Un entorno de desarrollo integrado (IDE) es una aplicación que proporciona a los desarrolladores todas las herramientas necesarias para escribir, editar, probar y depurar código dentro de un mismo entorno. Existen varias opciones que se adaptan a las necesidades de distintos tipos de proyectos, desde aplicaciones sencillas hasta desarrollos más complejos. A continuación, se presentarán algunos de los IDEs más utilizados por la comunidad Python.

IDLE

IDLE es el entorno de desarrollo que se instala de forma predeterminada junto con Python. Se trata de una herramienta ligera y de uso sencillo, pensada principalmente para quienes están iniciando en el aprendizaje de este lenguaje. Está construido sobre Tkinter, la biblioteca gráfica de Python, lo que le permite ofrecer una interfaz simple pero funcional. Entre sus características más representativas se encuentran:

- Compatible con varios sistemas operativos, como Windows, macOS y Linux.
- Una interfaz con una ventana de shell que permite ejecutar comandos y mostrar resultados.
- Un editor de código con resaltado de sintaxis y autocompletado.
- Un depurador integrado para facilitar el análisis de código.

A diferencia de otros entornos de desarrollo, IDLE no requiere configuraciones adicionales ni descargas externas, ya que se instala de manera automática junto con Python desde su página oficial (<https://www.python.org>). lo que lo convierte en una opción muy conveniente para quienes comienzan con Python. Una vez instalado Python, IDLE puede ser iniciado fácilmente

desde el menú de inicio en Windows o desde la terminal en macOS y Linux.

PyCharm

Creado por la compañía checa JetBrains, PyCharm se ha consolidado como uno de los entornos más utilizados por la comunidad de Python. Está disponible en dos ediciones: una gratuita, denominada Community Edition, que resulta adecuada para proyectos de menor escala, y una versión de pago (Professional Edition), orientada al desarrollo de aplicaciones más complejas. Entre sus principales funciones destacan:

- Compatible con varios sistemas operativos, como Windows, macOS y Linux.
- Autocompletado de código y análisis en tiempo real.
- Herramientas de depuración, pruebas y corrección de errores.
- Soporte para marcos web como Django y Flask.
- Integración con otros lenguajes de programación como JavaScript y TypeScript.
- Soporte para bibliotecas científicas como Matplotlib y NumPy.
- Posibilidad de trabajar con máquinas virtuales remotas.

Para descargar PyCharm, puedes visitar el sitio web oficial de JetBrains: <https://www.jetbrains.com/pycharm/>. Allí encontrarás tanto la versión gratuita como la versión profesional, según las necesidades de tu proyecto.

Spyder

Spyder es un entorno de desarrollo de código abierto que ha ganado gran reconocimiento en el ámbito científico y en el análisis de datos. Está diseñado para ofrecer un espacio de trabajo integral, combinando herramientas de programación con utilidades propias del análisis científico. Entre sus principales características se encuentran:

- Compatible con varios sistemas operativos, como **Windows, macOS y Linux**.
- Soporte para varios lenguajes de programación.
- Consola interactiva de **IPython**.
- Herramientas para depuración y visualización de datos.
- Integración con bibliotecas científicas como **SciPy, Matplotlib y NumPy**.
- Capacidad para explorar variables y visualizar documentación en tiempo real.

Puedes descargar Spyder desde su página oficial: <https://www.spyder-ide.org/>, o instalarlo fácilmente mediante el gestor de paquetes **Anaconda**, que incluye Spyder y otras herramientas útiles para la ciencia de datos.

Atom

Atom, desarrollado por GitHub, es un editor de código gratuito que destaca por su alto nivel de personalización. Aunque fue concebido como una herramienta multipropósito compatible con diversos lenguajes de programación, también resulta muy eficiente para trabajar con Python. Entre las funciones que lo caracterizan se encuentran:

- Compatible con varios sistemas operativos, como **Windows, macOS y Linux**.
- Instalación sencilla y creación de nuevos paquetes.
- Integración directa con **GitHub** para almacenar proyectos de manera centralizada.
- Autocompletado rápido y búsqueda eficiente de archivos y proyectos.
- Personalización completa de la interfaz.

Puedes descargar Atom directamente desde su página oficial: <https://atom.io/>.

Visual Studio Code (VSCode)

Visual Studio Code, desarrollado por Microsoft, es un editor de código abierto que se ha posicionado como una de las herramientas más utilizadas por programadores de distintos lenguajes. Si bien no se trata de un IDE en sentido estricto, su funcionalidad puede ampliarse mediante extensiones que permiten adaptarlo a las necesidades del desarrollo en Python. Entre sus características más destacadas se encuentran:

- Compatible con varios sistemas operativos, como Windows, macOS y Linux.
- Soporte para depuración, autocompletado y linting en Python.
- Amplia comunidad y gran número de extensiones disponibles.
- Integración con Git y plataformas de control de versiones.
- Soporte para realizar pruebas unitarias y depuración remota

Puedes descargar Visual Studio Code desde su página oficial: <https://code.visualstudio.com/>



¿Qué son los SDK en Python?

Un Kit de Desarrollo de Software (SDK) se define como un conjunto de herramientas y recursos diseñados para facilitar la creación de aplicaciones en un lenguaje de programación determinado. Por lo general, estos kits están asociados a plataformas o sistemas operativos específicos y proporcionan a los desarrolladores lo necesario para acelerar y optimizar el proceso de desarrollo.

En el caso de Python, los SDK suelen incluir componentes fundamentales como bibliotecas, fragmentos de código de ejemplo y documentación especializada. Estos elementos permiten no solo escribir programas de manera más rápida, sino también probarlos e implementarlos con mayor eficacia y confiabilidad.

1.6. Entornos interactivos (análisis de datos y experimentación)

En el desarrollo y análisis de datos con Python, los entornos interactivos juegan un papel fundamental al permitir la ejecución de código en fragmentos, la visualización de datos en tiempo real y la integración de herramientas de análisis estadístico y machine learning. A diferencia de los IDE tradicionales, estos entornos están diseñados para la experimentación y la iteración rápida, facilitando el trabajo de científicos de datos, analistas y desarrolladores. A continuación, se presentan algunos de los entornos interactivos más utilizados en Python, junto con sus principales características y aplicaciones.

Jupyter Notebook

Jupyter Notebook se ha consolidado como uno de los entornos interactivos más utilizados en programación con Python, especialmente en el ámbito del análisis de datos y la investigación científica. Su principal característica es la posibilidad de ejecutar fragmentos de código en celdas independientes, lo que favorece la experimentación, la validación de ideas y el desarrollo progresivo de proyectos. No obstante, también se destaca que se integra con bibliotecas como Pandas, NumPy y Matplotlib para análisis y visualización de datos.

Casos de uso:

- Ciencia de datos y machine learning.
- Creación de reportes interactivos.
- Desarrollo y prueba de modelos de análisis estadístico.

JupyterLab

representa la versión más avanzada de Jupyter Notebook, ofreciendo una interfaz moderna, flexible y modular. Puedes obtener más información en su sitio oficial: <https://jupyter.org/>

Características principales:

- Permite trabajar con varios archivos de forma simultánea, incluyendo *notebooks*, terminales, documentos de texto y scripts.
- Extensible mediante complementos y personalización de la interfaz.
- Mejora en la organización y gestión de proyectos grandes.

Casos de uso:

- Desarrollo colaborativo en ciencia de datos.
- Análisis de grandes volúmenes de información.
- Trabajo con datos en la nube y en entornos empresariales.

Google Colab

Google Colab es una herramienta en línea creada por Google que funciona de manera similar a Jupyter Notebook, pero alojada en la nube. Gracias a esta plataforma, es posible ejecutar código en Python sin instalar programas en el computador, ya que todo se gestiona directamente desde el navegador. El acceso es gratuito y se realiza a través del sitio oficial: <https://colab.google/>

Características principales:

- Ejecución en la nube sin instalación previa.
- Soporte gratuito para GPUs y TPUs.
- Integración con Google Drive para almacenamiento.
- Compartición fácil de notebooks.

Casos de uso:

- Machine learning y deep learning con TensorFlow y PyTorch.
- Trabajo colaborativo en equipos de datos.
- Experimentación sin consumo de recursos locales.

Apache Zeppelin

Apache Zeppelin es un entorno interactivo diseñado principalmente para el análisis de grandes volúmenes de datos y la ejecución de procesos distribuidos. Se integra de forma nativa con Apache Spark, aunque también es compatible con otras tecnologías de procesamiento de datos, lo que lo convierte en una herramienta flexible para proyectos de big data. Puedes obtener más información en su sitio oficial: <https://zeppelin.apache.org/>

Características principales:

- Soporte para múltiples lenguajes: Python, Scala, SQL, etc.
- Integración con herramientas de big data como Hadoop y Spark.
- Paneles visuales interactivos y colaboración en tiempo real.

Casos de uso:

- Análisis de grandes volúmenes de datos.
- Generación de informes visuales interactivos.
- Trabajo en entornos empresariales con procesamiento distribuido



¿Cuál entorno elegir?

Seleccionar el entorno adecuado depende de tus necesidades y nivel de experiencia. Aquí algunas recomendaciones según el uso:

- ☒ **Para principiantes** → IDLE
- ☒ **Para desarrollo profesional** → PyCharm, Spyde, VS Code
- ☒ **Para ciencia de datos** → JupyterLab, Google Colab, Zeppelin

1.7. Sintaxis y programación básica en Python

Para crear programas funcionales y eficientes en Python, es indispensable conocer sus fundamentos principales. Entre ellos se encuentran las variables, los tipos de datos, los operadores, las estructuras de control, los bucles y las funciones. Estos recursos permiten organizar el código de forma modular y reutilizable, lo que facilita tanto la gestión de la información como la automatización de procesos. El dominio de estas nociones iniciales constituye la base para aprovechar al máximo el potencial de Python en áreas como el análisis de datos y la inteligencia artificial.

Variables y tipos de datos

Una variable puede entenderse como un contenedor que guarda un valor en memoria para ser utilizado posteriormente en el programa. En Python, no es necesario especificar de antemano el tipo de dato que almacenará una variable, ya que este se asigna de manera automática en tiempo de ejecución. Esta característica, conocida como tipado dinámico, otorga flexibilidad y agilidad al momento de programar.

Figura 2

Principales tipos de datos en Python

Tipo de Dato	Descripción	Ejemplo
int (entero)	Números enteros	x = 10
float (decimal)	Números con punto decimal	pi = 3.14
str (cadena)	Texto entre comillas	nombre = "Ana"
bool (booleano)	True o False	activo = True
list (lista)	Conjunto de valores modificable	numeros = [1, 2, 3]
tuple (tupla)	Conjunto de valores inmutable	coordenadas = (10, 20)
dict (diccionario)	Clave-valor	persona = {"nombre": "Juan", "edad": 25}
set (conjunto)	Colección única de valores	colores = {"rojo", "verde", "azul"}

Figura 3

Ejemplo básico del uso de variables

```
python
# Variables numéricas
edad = 25 # int
altura = 1.75 # float

# Cadenas de texto
nombre = "Carlos"

# Booleanos
es_mayor = edad > 18 # True

# Listas
frutas = ["manzana", "pera", "uva"]
print(frutas[0]) # "manzana"

# Diccionarios
persona = {"nombre": "Ana", "edad": 30}
print(persona["nombre"]) # "Ana"
```

Operadores aritméticos

Los operadores aritméticos en Python permiten realizar operaciones matemáticas elementales como suma, resta, multiplicación, división y exponenciación. Su uso no se limita únicamente a cálculos numéricos, sino que también resulta fundamental en la manipulación de datos y en la construcción de algoritmos que requieren procesar información a través de operaciones básicas. Gracias a estos operadores, los programadores pueden implementar desde simples expresiones matemáticas hasta procesos más complejos relacionados con análisis de datos o simulaciones computacionales.

Figura 4

Operadores aritméticos

Operador	Descripción	Ejemplo
+	Suma	5 + 3 → 8
-	Resta	10 - 4 → 6
*	Multiplicación	6 * 3 → 18
/	División	10 / 3 → 3.33
//	División entera	10 // 3 → 3
%	Módulo (residuo)	10 % 3 → 1
**	Potencia	2 ** 3 → 8

Figura 5

Ejemplo básico del uso de operadores aritméticos

```
python
a = 10
b = 3
print(a + b)    # 13
print(a // b)   # 3 (División entera)
print(a % b)    # 1 (Residuo)
print(a ** b)   # 1000 (10^3)
```

Operadores relacionales

En Python, los operadores relacionales permiten establecer comparaciones entre dos valores para determinar su relación. El resultado de estas comparaciones es un valor booleano, es decir, True si la condición se cumple o False en caso contrario. Este tipo de operadores resulta fundamental en la lógica de los programas, pues posibilita la toma de decisiones automáticas. Con frecuencia, se aplican dentro de estructuras de control como if, while o for, ya que permiten definir el camino que seguirá la ejecución del código en función de determinadas condiciones.

Figura 6

Operadores relacionales

Operador	Descripción	Ejemplo
==	Igual a	5 == 5 → True
!=	Distinto de	5 != 3 → True
>	Mayor que	10 > 5 → True
<	Menor que	2 < 8 → True
>=	Mayor o igual que	7 >= 7 → True
<=	Menor o igual que	3 <= 5 → True

Figura 7

Ejemplo básico del uso de operadores relacionales

```
python
x = 10
y = 20
print(x < y)    # True
print(x == y)   # False
```

Operadores Lógicos

En Python, los operadores lógicos se emplean para unir y evaluar múltiples condiciones dentro de una misma expresión booleana. Gracias a ellos, es posible construir comprobaciones más complejas que van más allá de una sola comparación. Estos operadores desempeñan un papel esencial en el control del flujo de los programas, ya que permiten definir decisiones y repetir procesos en función de varios criterios al mismo tiempo.

Figura 8

Operadores lógicos

Operador	Descripción	Ejemplo
and	Devuelve True si ambas condiciones son True	(5 > 3) and (10 > 5) → True
or	Devuelve True si al menos una condición es True	(5 > 3) or (10 < 5) → True
not	Invierte el resultado lógico	not (5 > 3) → False

Figura 9

Ejemplo básico del uso de operadores lógicos

```
python
es_adulto = edad >= 18 and altura > 1.5
print(es_adulto) # True
```

Estructuras de Control (if, elif, else)

En Python, las estructuras condicionales *if*, *elif* y *else* resultan indispensables para guiar el flujo de ejecución de un programa. Su función principal es permitir que el código se comporte de manera distinta según el cumplimiento o no de ciertas condiciones. Gracias a estas sentencias, los programas pueden adaptarse a diferentes escenarios de entrada y responder de manera flexible, lo que aporta dinamismo y mayor capacidad de adaptación en la resolución de problemas.

Figura 10

Ejemplo básico del uso del if

```
python
edad = 18

if edad >= 18:
    print("Eres mayor de edad")
elif edad >= 13:
    print("Eres un adolescente")
else:
    print("Eres un niño")
```

Bucles (for, while)

En Python, los bucles son estructuras que facilitan la repetición de instrucciones sin necesidad de escribirlas varias veces. El bucle *for* se emplea cuando se conoce de antemano el número de iteraciones, por ejemplo, al recorrer listas, cadenas o rangos numéricos. En cambio, el bucle *while* ejecuta un bloque de código mientras se cumpla una condición, lo cual resulta útil en situaciones donde no se puede determinar previamente la cantidad de repeticiones. Ambos mecanismos son esenciales para optimizar el trabajo del programador, ya que permiten automatizar procesos, reducir redundancias y mantener un código más claro y eficiente.

Figura 11

Ejemplo básico del uso del for

```
python

# Bucle for para iterar a través de los números del 1 al 10
for i in range(1, 11):
    print(f"Número: {i}")

# Bucle while para verificar si el número es par
while i % 2 == 0:
    print("Este número es par.")
    break # Salir del bucle while después de verificar que es par
```

Funciones en Python

En Python, las funciones constituyen bloques de código diseñados para ejecutar tareas específicas de manera organizada y reutilizable. Al crear una función, es posible agrupar varias instrucciones bajo un identificador único y llamarlas cada vez que sea necesario, evitando la repetición innecesaria de código. Estas pueden recibir parámetros que permiten trabajar con diferentes valores y, a su vez, devolver resultados mediante la instrucción *return*. El uso de funciones no solo contribuye a una mayor claridad y legibilidad en los programas, sino que también favorece la modularidad, facilitando el mantenimiento y la ampliación de los proyectos de software.

Figura 12

Ejemplo básico del manejo de funciones

```
python
# Definición de una función con múltiples parámetros
def calcular_area_rectangulo(base, altura):
    """
    Esta función calcula el área de un rectángulo.
    Parámetros:
    - base: la base del rectángulo
    - altura: la altura del rectángulo
    Retorna:
    - El área del rectángulo (base * altura)
    """
    return base * altura

# Llamada a la función con valores específicos
area = calcular_area_rectangulo(5, 3)
print(f"El área del rectángulo es: {area}")

# Función Lambda (anónima) para calcular el cuadrado de un número
cuadrado = lambda x: x ** 2

# Llamada a la función Lambda
resultado_cuadrado = cuadrado(4)
print(f"El cuadrado de 4 es: {resultado_cuadrado}")
```

Ejercicio: Análisis de Notas de Estudiantes

El objetivo de este programa es practicar la aplicación de conceptos básicos de Python a través de un caso sencillo. La aplicación desarrollada deberá:

1. Solicitar al usuario el nombre y las calificaciones de un estudiante.
2. Calcular el promedio de dichas calificaciones.
3. Determinar, con base en el promedio, si el estudiante ha aprobado o reprobado.
4. Utilizar una función para realizar los cálculos de manera modular.
5. Incorporar un módulo de Python que permita formatear y presentar los resultados de forma más clara.

Figura 13

Código fuente de la aplicación

```
python

# Importamos el módulo math para redondear el promedio
import math

# Definimos una función para calcular el promedio de notas
def calcular_promedio(notas):
    promedio = sum(notas) / len(notas) # Suma todas las notas y divide por la cantidad
    return round(promedio, 2) # Redondea a 2 decimales

# Solicitamos información del estudiante
nombre = input("Ingrese el nombre del estudiante: ")

# Capturamos tres notas y las convertimos a una lista de números
notas = [float(input(f"Ingrese la nota {i+1}: ")) for i in range(3)]

# Calculamos el promedio
promedio = calcular_promedio(notas)

# Determinamos si aprobó (mínimo 3.0 para aprobar)
estado = "Aprobado" if promedio >= 3.0 else "Reprobado"

# Mostramos los resultados
print("\n--- Resultado ---")
print(f"Estudiante: {nombre}")
print(f"Notas: {notas}")
print(f"Promedio: {promedio}")
print(f"Estado: {estado}")
```

Resumen paso a paso:

1. Importar módulo: incluir `import math` para emplear sus funciones de redondeo en los resultados numéricos.
2. Definir función: crear una rutina que reciba una lista de calificaciones, calcule su promedio y aplique el redondeo correspondiente.
3. Capturar datos: solicitar al usuario el nombre del estudiante y tres calificaciones, convirtiendo cada entrada a tipo `float`.
4. Calcular y evaluar: obtener el promedio de las notas y comprobar si es mayor o igual a 3.0; en ese caso clasificar como aprobado, en caso contrario como reprobado.
5. Mostrar resultado: presentar en pantalla el nombre, las calificaciones ingresadas, el promedio (formateado/redondeado) y el estado final (aprobado/reprobado).

Figura 14

Ejemplo de ejecución

```
Ingrese el nombre del estudiante: Juan Pérez
Ingrese la nota 1: 4.5
Ingrese la nota 2: 3.8
Ingrese la nota 3: 2.9

--- Resultado ---
Estudiante: Juan Pérez
Notas: [4.5, 3.8, 2.9]
Promedio: 3.73
Estado: Aprobado
```

Capítulo 2

Organización del Código y Herramientas de Desarrollo en Python

Python es un lenguaje ampliamente utilizado en la analítica de datos debido a su simplicidad y la gran cantidad de bibliotecas especializadas. Para desarrollar programas eficientes y escalables, es fundamental seguir una estructura organizada. La estructura del código no solo influye en su claridad y reutilización, sino que también facilita la colaboración entre desarrolladores y la integración con herramientas de control de versiones.

Este capítulo aborda los principios fundamentales para organizar y estructurar el código en Python, abordando temas clave como la división en módulos y paquetes, y las convenciones de estilo recomendadas. Se profundiza en la gestión de dependencias con entornos virtuales, así como el manejo de archivos y bases de datos. También se introduce la creación de interfaces gráficas, herramientas fundamentales para desarrollar aplicaciones completas en Python. Al dominar estos conceptos, los lectores podrán escribir código modular, eficiente y profesional.



2.1. Organización del código en scripts y módulos

En Python, la organización y estructuración del código es fundamental para garantizar su escalabilidad, legibilidad y mantenimiento. Dependiendo de la complejidad del proyecto, la organización puede ser muy simple o más detallada. A continuación, profundizamos en las distintas formas de estructurar un programa en Python.

Script

Un script en Python es un archivo único con la extensión .py que contiene un conjunto de instrucciones que Python puede ejecutar directamente. Los scripts son adecuados para proyectos sencillos o tareas individuales que no requieren demasiada modularización. Un script puede incluir funciones, clases y código ejecutable, y generalmente se utiliza para tareas de automatización o análisis simples.

Ejemplo básico de script en python:

```
python

# analisis_datos.py
import pandas as pd

# Cargar Los datos
datos = pd.read_csv('datos.csv')

# Analizar los datos
resumen = datos.describe()

# Imprimir el resumen
print(resumen)
```

El script se ejecuta de manera secuencial, y su estructura es bastante simple. Sin embargo, a medida que los proyectos crecen en tamaño y complejidad, los scripts pueden volverse difíciles de mantener. Es por esto que, para proyectos más grandes, se recomienda organizar el código en **módulos** y **paquetes**.

Módulo

Un módulo en Python es simplemente un archivo .py que contiene definiciones de funciones, clases y variables, además de código ejecutable. La idea de un módulo es que proporciona una unidad de funcionalidad que se puede reutilizar en otras partes de tu proyecto. Los módulos permiten dividir el código en bloques más pequeños y lógicos, facilitando su mantenimiento y actualización.

Por ejemplo, si tienes un conjunto de funciones para limpiar datos, puedes crear un archivo independiente llamado limpieza.py y definir todas las funciones de limpieza allí.

Ejemplo básico de módulo:

```
python

# limpieza.py
import pandas as pd

def eliminar_valores_nulos(datos):
    return datos.dropna()

def convertir_a_lowercase(datos, columna):
    datos[columna] = datos[columna].str.lower()
    return datos
```

En este ejemplo, el módulo `limpieza.py` contiene dos funciones: una para eliminar los valores nulos y otra para convertir texto a minúsculas. Estos módulos se pueden importar y reutilizar en otros scripts o módulos dentro del proyecto.

Paquete

Un paquete es una forma de organizar múltiples módulos relacionados en una carpeta. Un paquete es, esencialmente, un directorio que contiene varios archivos de Python, y debe incluir un archivo especial llamado `__init__.py` que Python utiliza para reconocer el directorio como un paquete.

Los paquetes permiten organizar proyectos más complejos en subcomponentes que se pueden desarrollar y mantener de manera independiente. Además, los paquetes pueden incluir subpaquetes, lo que da lugar a una estructura jerárquica que facilita aún más la organización de los módulos.

Ejemplo básico de estructura de proyecto:

```
mi_proyecto/           # Raíz del proyecto
├── analisis_datos.py   # Archivo para análisis de datos
├── limpieza.py         # Archivo para limpiar los datos
└── datos/              # Directorio/paquete que agrupa varios módulos relacionados
    ├── __init__.py     # Inicialización del paquete "datos"
    ├── cargar.py        # Módulo para cargar los datos
    ├── transformar.py   # Módulo para transformar los datos
    └── visualizar.py    # Módulo para visualizar los datos
```

En esta estructura, el directorio `datos/` es un paquete que agrupa varios módulos relacionados con la manipulación de datos. El archivo `__init__.py` es necesario para que Python reconozca que `datos/` es un paquete, aunque puede estar vacío o contener código de

inicialización del paquete.

Cada uno de los módulos dentro del paquete puede tener una función o responsabilidad específica. Por ejemplo:

- **cargar.py** podría contener funciones para cargar datos desde diferentes fuentes.
- **transformar.py** podría tener funciones para transformar esos datos, como realizar limpiezas o agregaciones.
- **visualizar.py** podría contener funciones para generar gráficos o informes.

Para usar un módulo de este paquete en tu script o en otro módulo, solo necesitas importarlo de la siguiente manera:

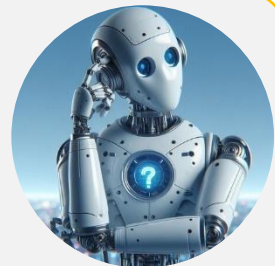
```
python

from datos import cargar, transformar

# Usar las funciones del módulo
datos = cargar.cargar_datos('archivo.csv')
datos_limpios = transformar.limpiar_datos(datos)
```

Sabías que ...

La principal ventaja de organizar tu proyecto en scripts, módulos y paquetes es que facilita la reutilización del código, la colaboración en equipo y la gestión de proyectos complejos. Cada componente del código se puede desarrollar y probar independientemente, lo que reduce los errores y mejora la eficiencia.



2.2. Convenciones de estilo en Python (PEP 8)

La **PEP 8** Python Enhancement Proposal 8 (Propuesta de Mejora de Python 8) es la guía de estilo oficial para escribir código en Python. Su propósito es asegurar que el código sea **legible, consistente y mantenible**, no solo para el programador que lo escribe, sino también para los demás que lo leen o lo mantienen a lo largo del tiempo. A continuación, se relacionan las convenciones de estilo más importantes recomendadas por PEP 8.

Indentación: Usar 4 espacios por nivel de indentación

La indentación es crucial en Python, ya que define los bloques de código. En lugar de usar tabulaciones, PEP 8 recomienda utilizar **4 espacios** por nivel de indentación. Esto mejora la legibilidad y garantiza que el código sea consistente.

Ejemplo:

```
python

def ejemplo_funcion():
    if condicion:
        # Bloque de código dentro del if
        hacer_algo()
    else:
        # Bloque de código dentro del else
        hacer_algo_mas()
```

Usar espacios en lugar de tabulaciones previene problemas de compatibilidad entre editores de texto y asegura que el código sea consistente en cualquier entorno. La convención de 4 espacios se adopta porque es un estándar ampliamente aceptado en la comunidad Python y está muy extendido.

Nombres de variables: Usar snake_case

Snake_case es un estilo de nomenclatura en el que las palabras de un nombre de variable se separan por guiones bajos (_), y es comúnmente utilizado en lenguajes como Python para variables, funciones y métodos. En contraste, camelCase y PascalCase son estilos alternativos de nomenclatura.

- **CamelCase** (también conocido como "notación de camello") capitaliza la primera letra de cada palabra, excepto la primera. Por ejemplo: miVariableEjemplo.
- **PascalCase** es similar al camelCase, pero en este caso, la primera letra de cada palabra, incluida la primera, también se capitaliza. Ejemplo: MiVariableEjemplo.

Ejemplo básico nombres de variables:

```
python

# Correcto
datos_limpios = limpiar_datos(df)

# Incorrecto
datoslimpios = limpiarDatos(df)
```

Este estilo de nombres mejora la legibilidad al ser claro y uniforme. Además, es compatible con la convención general que se sigue en la comunidad de Python.

Espaciado: Dejar un espacio antes y después de operadores

Es una buena práctica dejar un espacio antes y después de los operadores como =, +, -, *, and, or, entre otros. Esto mejora la legibilidad, haciendo que el código sea más claro y fácil de seguir.

Ejemplo básico de legibilidad:

```
python

# Correcto
a = b + c

# Incorrecto
a=b+c
```

Los espacios entre los operadores hacen que el código se vea más limpio y organizado. Sin ellos, el código puede parecer apretado y difícil de leer, lo que puede dificultar la comprensión rápida.

Comentarios: Usar # para comentarios en línea y """docstrings""" para documentar funciones

Los comentarios son esenciales para explicar el propósito y la lógica del código. PEP 8 establece que deben usarse dos tipos de comentarios:

- **Comentarios en línea:** Se utilizan para explicar una línea específica de código. Estos comentarios deben comenzar con el símbolo # y deben estar precedidos por al menos un espacio.
- **Docstrings:** Son comentarios más detallados que se usan para documentar funciones, clases y módulos. Se definen utilizando triple comillas ("""") y deben proporcionar una descripción clara de lo que hace la función, clase o módulo.

Ejemplo básico del uso de comentarios:

```
python

x = 10 # Este es el valor inicial de x

def limpiar_datos(df):
    """Elimina valores nulos y columnas innecesarias de un DataFrame."""
    df = df.dropna() # Elimina filas con valores nulos
    df = df.drop(columns=['columna_inutil']) # Elimina columnas innecesarias
    return df
```

Acceder a los docstrings: Para ver la descripción de una función documentada con un docstring, puedes utilizar la función `help()` o acceder al atributo `__doc__`. Por ejemplo:

```
python

help(limpiar_datos) # Muestra el docstring de la función
print(limpiar_datos.__doc__) # Imprime el docstring de la función
```

Los comentarios y docstrings son cruciales para que otros desarrolladores (o para ti mismo) puedan entender rápidamente el propósito del código sin tener que analizar cada línea. Un buen comentario o docstring puede ser la diferencia entre un código claro y un código confuso. Además, los docstrings facilitan el uso de herramientas de documentación automática como **Sphinx**, **pdoc**, **PyDoc** entre otros, lo que permite generar documentación de proyectos de manera eficiente.

Ejemplo básico de una función bien escrita según PEP 8:

```
python

def limpiar_datos(df):
    """Elimina valores nulos y columnas innecesarias."""
    # Elimina las filas con valores nulos
    df = df.dropna()

    # Elimina columnas que no se utilizan en el análisis
    df = df.drop(columns=['columna_inutil'])

    return df
```

Aspectos importantes en este ejemplo:

- **Nombres de variables:** df es un nombre claro y conciso para un DataFrame (aunque se recomienda usar nombres más descriptivos cuando sea necesario).
- **Comentarios:** Se usan comentarios para explicar el propósito de las líneas de código, sin que sean excesivos.
- **Docstring:** La función tiene un docstring que explica qué hace la función y qué parámetros recibe, lo que facilita la comprensión.

Estas son solo algunas de las recomendaciones de PEP 8. Para profundizar más en las mejores prácticas para escribir código en Python, puedes revisar la documentación oficial en el siguiente enlace [PEP 8 – Style Guide for Python Code](#)

2.3. Uso de entornos virtuales para la gestión de dependencias

Cuando desarrollamos proyectos en Python, es común trabajar con múltiples librerías de terceros. Sin embargo, instalar estas dependencias de manera global en el sistema puede generar conflictos entre versiones de paquetes usados en diferentes proyectos. Para evitar estos problemas y facilitar la portabilidad del código, se recomienda el uso de entornos virtuales.

Un entorno virtual en Python es un espacio aislado dentro del sistema donde se pueden instalar paquetes específicos sin afectar el resto del entorno o el sistema global de Python. Esto

permite:

- Evitar conflictos de versiones entre proyectos.
- Facilitar la reproducibilidad del código en otros entornos.
- Mantener un entorno limpio sin instalar paquetes innecesarios en el sistema global.

2.3.1. Pasos para crear un entorno virtual

Para crear un entorno virtual, debes seguir los siguientes pasos:

❶ Crear un entorno virtual

Para crear un entorno virtual, se usa el módulo `venv` de Python. Este módulo permite crear un entorno aislado, donde puedes gestionar las dependencias de tu proyecto sin afectar el sistema global de Python.

```
sh  
  
python -m venv mi_entorno
```

Esto generará una carpeta llamada `mi_entorno` que contendrá una copia aislada de Python y un espacio para instalar librerías.

❷ Activar el entorno

Dependiendo del sistema operativo, la activación del entorno se realiza de la siguiente manera:

- **En Windows** (Símbolo del sistema o PowerShell):

```
sh  
  
mi_entorno\Scripts\activate
```

- **En macOS/Linux** (Terminal):

```
sh  
  
source mi_entorno/bin/activate
```

Después de la activación, el nombre del entorno virtual aparecerá en el **prompt** de la terminal, indicando que el entorno está en uso.

❸ Instalar dependencias

Una vez activado el entorno, se pueden instalar paquetes de Python con `pip`, el gestor de paquetes de Python:

```
sh

pip install numpy pandas matplotlib
```

Esto instalará las librerías en el entorno virtual sin afectar el entorno global de Python en el sistema.

④ Guardar las dependencias en un archivo

Para documentar las librerías instaladas y sus versiones exactas, se recomienda usar freeze:

```
sh

pip freeze > requirements.txt
```

Esto creará un archivo requirements.txt con la lista de dependencias, lo que facilita su instalación en otro entorno.

⑤ Restaurar dependencias en otro entorno

Para reproducir el entorno en otra máquina o en otro momento, simplemente se ejecuta:

```
sh

pip install -r requirements.txt
```

Esto instalará todas las librerías con sus versiones especificadas en el archivo. Un archivo requirements.txt típico para proyectos de ciencia de datos puede verse así:

```
numpy==1.21.2
pandas==1.3.3
matplotlib==3.4.3
seaborn==0.11.2
scikit-learn==0.24.2
```

Este archivo asegura que todos los colaboradores de un proyecto usen **las mismas versiones** de cada paquete, evitando inconsistencias en el código.

Sabías que ...

Algunos de los beneficios del uso de Entornos Virtuales son:

- ✓ *Aislamiento: Cada proyecto tiene su propio entorno sin interferencias de otros.*
- ✓ *Compatibilidad: Permite replicar fácilmente el entorno en diferentes equipos o servidores.*
- ✓ *Orden y mantenimiento: Evita la instalación de paquetes innecesarios en el sistema global.*
- ✓ *Despliegue: Facilita el despliegue en producción o en otros entornos de trabajo.*





Visual Studio Code (VS Code)

En **Visual Studio Code**, puedes crear y activar automáticamente un entorno virtual desde la **Paleta de Comandos**, lo que simplifica aún más el proceso.

Creación Automática de un Entorno Virtual en VS Code:

- 1 Abre **Visual Studio Code** y asegúrate de que la extensión de **Python** esté instalada.
- 2 Ve al menú **Ver** → **Paleta de Comandos** (o usa Ctrl + Shift + P en Windows/Linux, Cmd + Shift + P en macOS).
- 3 Escribe y selecciona la opción: **"Python: Create Environment"** (*Crear Ambiente en Python*)
- 4 VS Code te pedirá que elijas el tipo de entorno: Selecciona venv para un entorno virtual.
- 6 VS Code **creará y activará automáticamente** el entorno virtual dentro de la carpeta del proyecto.
- 7 Una vez creado, el entorno virtual se activará en la terminal y se configurará automáticamente como el intérprete de Python para el proyecto.

2.4. Manejo de archivos y bases de datos

El manejo de archivos y bases de datos es esencial en el procesamiento de datos. Python proporciona herramientas y bibliotecas especializadas para trabajar con archivos estructurados como CSV, JSON y Excel, así como para conectarse a bases de datos SQL y NoSQL.

2.4.1. Lectura y escritura de archivos CSV, JSON y Excel

Python facilita la manipulación de archivos de datos estructurados mediante la biblioteca pandas, que permite leer y escribir archivos de distintos formatos de manera eficiente.

Manejo de Archivos CSV

Los archivos CSV (Comma-Separated Values) son ampliamente utilizados para el intercambio de datos. Pandas permite su fácil manipulación.

Ejemplo básico del manejo de archivos con Pandas:

```
python

import pandas as pd

# Leer un archivo CSV
df = pd.read_csv("datos.csv")

# Guardar DataFrame en CSV sin el índice
df.to_csv("datos_limpios.csv", index=False)
```

Este código en Python utiliza la librería de **pandas** para leer un archivo CSV (datos.csv) y almacenarlo en un **DataFrame**. Luego, guarda los datos procesados en un nuevo archivo (datos_limpios.csv) sin incluir la columna de índices, facilitando la manipulación y exportación de datos de forma eficiente.

Manejo de Archivos JSON

El formato JSON (JavaScript Object Notation) es muy utilizado en APIs y almacenamiento de datos estructurados. Python ofrece soporte nativo con el módulo json.

Ejemplo básico del manejo de archivos con Json:

```
python

import json

# Leer un archivo JSON
with open("datos.json", "r") as file:
    data = json.load(file)

# Guardar datos en formato JSON
with open("datos_salida.json", "w") as file:
    json.dump(data, file, indent=4)
```

Este código en Python utiliza el módulo json para leer un archivo JSON (datos.json) y cargar su contenido en la variable data. Luego, guarda estos datos en un nuevo archivo (datos_salida.json) con una indentación de 4 espacios, mejorando su legibilidad y facilitando su almacenamiento o intercambio.

Manejo de Archivos Excel

El formato Excel es común en entornos empresariales. Pandas permite leer y escribir archivos Excel mediante openpyxl o xlrd:

```
python
# Instalar librerías necesarias
# pip install openpyxl

# Leer un archivo Excel
df_excel = pd.read_excel("datos.xlsx", engine="openpyxl")

# Guardar en un nuevo archivo Excel
df_excel.to_excel("datos_procesados.xlsx", index=False, engine="openpyxl")
```

Este código en Python utiliza pandas para el análisis de datos, junto con openpyxl, para trabajar con archivos Excel en formato .xlsx. Primero, lee un archivo de Excel (datos.xlsx) y lo almacena en un DataFrame. Luego, guarda los datos en un nuevo archivo (datos_procesados.xlsx) sin incluir la columna de índices, permitiendo su manipulación y exportación eficiente. **Nota:** Es necesario instalar openpyxl para manejar archivos de Excel con extensión .xlsx

2.4.2. Conexión a bases de datos SQL y NoSQL

Python permite la conexión a diferentes tipos de bases de datos, tanto SQL (estructuradas) como NoSQL (no estructuradas).

Conexión a SQLite (Base de datos SQL)

SQLite es una base de datos SQL ligera, perfecta para aplicaciones pequeñas o de escritorio. Python tiene soporte nativo a través del módulo sqlite3:

Ejemplo básico de conexión a base de datos SQLite:

```
python
import sqlite3

# Conectar a la base de datos (crea el archivo si no existe)
conn = sqlite3.connect("mi_base.db")
cursor = conn.cursor()

# Crear una tabla si no existe
cursor.execute("""
CREATE TABLE IF NOT EXISTS datos (
    id INTEGER PRIMARY KEY,
    valor TEXT
)
""")

# Insertar datos
cursor.execute("INSERT INTO datos (id, valor) VALUES (?, ?)", (1, "dato1"))

# Guardar cambios y cerrar conexión
conn.commit()
conn.close()
```

Este código en Python utiliza `sqlite3` para gestionar una base de datos SQLite. Primero, establece una conexión con `mi_base.db` (creando el archivo si no existe) y obtiene un cursor para ejecutar comandos SQL. Luego, crea una tabla llamada `datos` si aún no existe, con una columna `id` como clave primaria y una columna `valor` de tipo texto. A continuación, inserta un registro en la tabla, guarda los cambios con `commit()` y cierra la conexión, asegurando una gestión eficiente de la base de datos.

Conexión a MongoDB (Base de datos NoSQL)

MongoDB es una base de datos NoSQL orientada a documentos. Se usa la librería `pymongo` para conectarse desde Python:

Ejemplo básico de conexión a base de datos MongoDB:

```
python

# Importar MongoClient de pymongo para conectarse a MongoDB
from pymongo import MongoClient

# Conectar a MongoDB en la dirección local (localhost) y el puerto predeterminado (27017)
client = MongoClient("mongodb://localhost:27017/")
# Seleccionar o crear la base de datos "mi_base"
db = client["mi_base"]
# Seleccionar o crear la colección "datos" dentro de la base de datos
coleccion = db["datos"]

# Insertar un documento en la colección
# El documento contiene un campo "id" con valor 1 y un campo "valor" con el texto "dato1"
coleccion.insert_one({"id": 1, "valor": "dato1"})

# Consultar todos los documentos en la colección e imprimirlos en la consola
for doc in coleccion.find():
    print(doc) # Cada documento se imprimirá como un diccionario de Python
# Cerrar la conexión con la base de datos
client.close() # Es una buena práctica cerrar la conexión después de su uso
```

Este código en Python utiliza **pymongo**, una librería para interactuar con bases de datos **MongoDB**. Primero, establece una conexión con un servidor local de MongoDB y accede a la base de datos `mi_base` y a la colección `datos`. Luego, inserta un documento con los campos `id` y `valor`, consulta todos los documentos almacenados e imprime los resultados. Finalmente, cierra la conexión para liberar recursos.

2.5. Creación de interfaces gráficas de usuarios (GUI) en Python

Las Interfaces Gráficas de Usuario (GUI) son herramientas esenciales para hacer que los programas sean más accesibles y fáciles de usar, especialmente cuando los usuarios no tienen experiencia en programación o cuando se requiere una interacción visual compleja. Python

proporciona varias bibliotecas para desarrollar estas interfaces gráficas, y la elección de una u otra depende de las necesidades específicas del proyecto. A continuación, se exploran algunas de las bibliotecas más populares para crear GUIs en Python:

Tkinter

Es la biblioteca estándar de Python para crear interfaces gráficas. Es ligera, fácil de aprender y perfecta para aplicaciones de escritorio sencillas o proyectos pequeños. Es la opción predeterminada para aquellos que no requieren interfaces visualmente complejas, pero que desean crear una GUI rápida y efectiva.

Características de Tkinter:

- Ligera y sencilla: Ideal para aplicaciones pequeñas.
- Fácil de aprender: Perfecta para principiantes.
- Integración con Python: No requiere instalación adicional, ya que viene preinstalada con Python.
- Widgets básicos: Incluye elementos esenciales como botones, etiquetas, cuadros de texto, marcos, menús y más.
- Basada en el paquete Tk: Utiliza el sistema de ventanas Tk, el cual es bastante popular por su simplicidad.
- Sistema basado en eventos: Utiliza un bucle de eventos (`mainloop()`) que permite responder a acciones del usuario como clics o escritura de texto.

Componentes principales de Tkinter:

- Ventana principal (Tk): Es la base de toda aplicación Tkinter.
- Widgets: Elementos gráficos como botones, etiquetas y campos de entrada.
- Gestión de diseño (`pack()`, `grid()`, `place()`): Métodos para organizar los widgets dentro de la ventana.
- Manejo de eventos: Permite capturar acciones del usuario y ejecutar funciones en respuesta.

Nota: Tkinter sigue siendo una excelente opción para proyectos pequeños o aplicaciones de escritorio sencillas. Si buscas interfaces más avanzadas y modernas, puedes explorar alternativas como **PyQt**, **PySide**.

Ejemplo básico de interfaz gráfica con Tkinter:

```
python
# Importación de librerías necesarias
import tkinter as tk
from tkinter import filedialog # Módulo para abrir el explorador de archivos
import pandas as pd # Librería para manipulación de datos
import matplotlib.pyplot as plt # Librería para visualización de gráficos

# Función para cargar un archivo CSV y visualizar sus datos
def cargar_archivo():
    # Abrir un cuadro de diálogo para seleccionar el archivo CSV
    archivo = filedialog.askopenfilename(filetypes=[("Archivos CSV", "*.csv")])

    # Verificar si se seleccionó un archivo
    if archivo:
        df = pd.read_csv(archivo) # Leer el archivo CSV en un DataFrame de Pandas
        print(df.head()) # Mostrar las primeras filas del DataFrame en la consola

        # Generar histogramas de las columnas numéricas del DataFrame
        df.hist()

        plt.show() # Mostrar los histogramas en pantalla

# Configuración de la ventana principal de la aplicación
root = tk.Tk()
root.title("Cargador de Datos") # Título de la ventana

# Definir tamaño de la ventana
ancho_ventana = 500
alto_ventana = 400

# Obtener dimensiones de la pantalla del usuario
ancho_pantalla = root.winfo_screenwidth()
alto_pantalla = root.winfo_screenheight()

# Calcular la posición para centrar la ventana en la pantalla
pos_x = (ancho_pantalla - ancho_ventana) // 2
pos_y = (alto_pantalla - alto_ventana) // 2

# Establecer el tamaño y la posición de la ventana
root.geometry(f"{ancho_ventana}x{alto_ventana}+{pos_x}+{pos_y}")

# Crear un botón que permite al usuario cargar un archivo CSV
btn_cargar = tk.Button(root, text="Cargar CSV", command=cargar_archivo)
btn_cargar.pack(pady=40) # Agregar espacio alrededor del botón

# Iniciar el bucle principal de la aplicación para que se mantenga abierta
root.mainloop()
```

Este código crea una ventana con un botón que carga un archivo CSV al hacer clic y muestra los datos en la terminal y un gráfico de barras con matplotlib.

Profundiza ...

Tkinter es la biblioteca estándar de Python para crear GUI. Es ligera, fácil de usar y perfecta para desarrollar aplicaciones de escritorio. Si deseas profundizar en el uso de Tkinter, te invito a explorar los siguientes recursos:

- ◆ Documento oficial de Tkinter → <https://docs.python.org/3/library/tkinter.html>
- ◆ Tkinter: Crea interfaces gráficas → <https://medium.com/@solidlucho/tkinter>



PyQt y PySide

Son bibliotecas que ofrecen una interfaz para Qt, un framework popular para el desarrollo de GUIs. Son más avanzadas que Tkinter y se utilizan para aplicaciones de escritorio más complejas que requieren un diseño más profesional y funcional.

Características de PyQt/PySide:

- Interfaz avanzada: Permite diseñar interfaces complejas y visualmente atractivas.
- Basado en Qt: Ofrece acceso a una amplia gama de widgets y herramientas para crear interfaces ricas.
- Mayor curva de aprendizaje: Requiere más conocimientos sobre programación de interfaces gráficas, pero ofrece mucha más flexibilidad y control.

Ejemplo básico de interfaz gráfica con PyQt:

```
python

# Importación de librerías necesarias para la interfaz gráfica
import sys # Manejo de argumentos del sistema
from PyQt5.QtWidgets import QApplication, QWidget, QPushButton # Componentes de PyQt5

# Función que se ejecuta al hacer clic en el botón
def on_click():
    print("¡Botón presionado!") # Muestra un mensaje en la consola cuando se presiona el botón

# Crear la aplicación principal de PyQt
app = QApplication(sys.argv) # Inicializa la aplicación y gestiona eventos del sistema

# Crear la ventana principal de la aplicación
window = QWidget() # Se define una ventana como un widget básico
window.setWindowTitle('Ejemplo PyQt') # Establece el título de la ventana
window.setGeometry(100, 100, 500, 400) # Define la posición (x, y) y dimensiones (ancho, alto)

# Crear un botón dentro de la ventana
btn = QPushButton('Presionar', window) # Crea un botón con el texto "Presionar"
btn.move(200, 160) # Ubica el botón en la ventana (posición x, y)

# Crear un botón dentro de la ventana
btn = QPushButton('Presionar', window) # Crea un botón con el texto "Presionar"
btn.move(200, 160) # Ubica el botón en la ventana (posición x, y)

# Conectar el clic del botón con la función on_click
btn.clicked.connect(on_click) # Cuando se presiona el botón, se ejecuta la función on_click

# Mostrar la ventana en pantalla
window.show() # Hace visible la ventana

# Iniciar el bucle de eventos de la aplicación (escucha interacciones del usuario)
sys.exit(app.exec_()) # Mantiene la aplicación en ejecución hasta que el usuario la cierre
```

Este ejemplo crea una ventana con un botón que, al hacer clic, imprime un mensaje en la consola.

Profundiza ...

PyQt es una de las bibliotecas más potentes para crear interfaces gráficas en Python. Permite desarrollar aplicaciones modernas con soporte multiplataforma. Si deseas profundizar en PyQt, te invito a explorar los siguientes recursos:

- Página oficial de PyQt → <https://www.riverbankcomputing.com/software/pyqt>
- Curso Python + PyQT → <https://youtu.be/Oy5saLTejtY?list>



Streamlit

Es una biblioteca moderna que facilita la creación de aplicaciones web interactivas para análisis de datos y machine learning. Es especialmente popular entre los científicos de datos y analistas que desean crear visualizaciones y aplicaciones rápidas sin tener que preocuparse por el diseño de la interfaz.

Características de Streamlit:

- Interactividad: Permite crear dashboards interactivos y aplicaciones web de forma rápida.
- Orientado a la analítica de datos: Ideal para la visualización y manipulación de datos en tiempo real.
- Simplicidad: No requiere experiencia previa en desarrollo web, ya que la sintaxis es sencilla y rápida.

Ejemplo básico de una aplicación web con Streamlit:

```
python

# Importar librerías necesarias
import streamlit as st # Streamlit para crear la aplicación web interactiva
import pandas as pd # Pandas para manejar y procesar datos CSV

# Título de la aplicación
st.title("Visualización de Datos") # Establece el título de la aplicación web

# Permite al usuario subir un archivo CSV desde la interfaz
archivo = st.file_uploader("Cargar archivo CSV", type=["csv"])

# Verificar si se ha subido un archivo
if archivo:
    # Leer el archivo CSV en un DataFrame de Pandas
    df = pd.read_csv(archivo)

    # Mostrar las primeras filas del DataFrame en la interfaz
    st.write(df.head())

    # Mostrar un gráfico de barras con las columnas numéricas del DataFrame
    st.bar_chart(df.select_dtypes(include='number'))
```

Este código crea una aplicación web con Streamlit para cargar un archivo CSV, mostrar

sus primeras filas y un gráfico de barras interactivo basado en los datos numéricos.

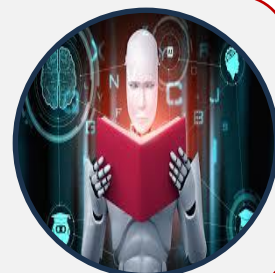
Como ejecutar el código:

Para ejecutar este código, solo se debe guardar en un archivo .py y luego ejecutar: `streamlit run archivo.py`

Profundiza ...

Streamlit es una de las herramientas más populares para crear aplicaciones web interactivas con Python de manera sencilla y rápida. Si quieres aprender más sobre Streamlit, te invito a explorar los siguientes recursos:

- *Página oficial de Streamlit* → <https://streamlit.io>
- *Introducción a Streamlit* → <https://www.youtube.com/watch?v=bssw63IiSxc>



Dash

Es un framework de Python diseñado para la creación de aplicaciones web interactivas enfocadas en el análisis y visualización de datos. Es ampliamente utilizado por científicos de datos y analistas que necesitan construir dashboards sin conocimientos avanzados de desarrollo web.

Características de Dash:

- **Interactividad:** Facilita la creación de dashboards dinámicos con gráficos y controles personalizados.
- **Basado en Flask y React:** Combina la flexibilidad de Flask con la potencia de React para interfaces avanzadas.
- **Escalabilidad:** Permite desarrollar desde prototipos simples hasta aplicaciones empresariales complejas.

Comparativo Interfaces Graficas de Usuario (GUI):

Biblioteca	Tipo de Aplicación	Usabilidad Web
PyQt / PySide	Aplicaciones de escritorio (GUI)	✗ No compatible con navegadores web
Tkinter	Aplicaciones de escritorio ligeras	✗ No compatible con navegadores web
Streamlit / Dash	Aplicaciones web interactivas	✓ 100% enfocadas en web

En el próximo capítulo, profundizaremos en Dash, explorando su estructura, componentes clave y el desarrollo de aplicaciones web interactivas. Dash es ampliamente utilizado para la creación de aplicaciones web robustas y altamente personalizables en ciencia de datos. A diferencia de Streamlit, diseñado para la simplicidad y la rápida creación de prototipos, Dash ofrece mayor control sobre la interactividad, el diseño y la integración con otros frameworks como Flask. Su enfoque basado en callbacks permite actualizar componentes sin recargar toda la aplicación, lo que lo convierte en una opción ideal para aplicaciones empresariales y proyectos escalables, donde la eficiencia y la flexibilidad son fundamentales.

Capítulo 3

Desarrollo y despliegue de aplicaciones web analíticas con Python

En la era de los datos, la visualización interactiva se ha convertido en una herramienta fundamental para la toma de decisiones. Dash, un framework basado en Flask, React y Plotly, permite el desarrollo de aplicaciones web interactivas en Python sin necesidad de conocimientos avanzados en desarrollo web. Su enfoque declarativo y su integración nativa con bibliotecas como Pandas y NumPy lo hacen ideal para la creación de dashboards dinámicos y personalizados, facilitando la exploración y el análisis de información en tiempo real.

En este capítulo, exploraremos el proceso completo de construcción de aplicaciones web interactivas con Dash, desde el diseño de la interfaz hasta su despliegue en la nube. Aprenderemos a estructurar aplicaciones modulares, gestionar la interactividad mediante callbacks y conectar nuestras aplicaciones con bases de datos y APIs externas. Además, abordaremos las mejores prácticas para garantizar su correcto funcionamiento en entornos de producción. Al finalizar, los estudiantes estarán capacitados para desarrollar y desplegar aplicaciones web interactivas de manera eficiente.



3.1. Python y su impacto en el desarrollo web

Python se ha consolidado como un lenguaje clave en el desarrollo web gracias a su simplicidad, versatilidad y amplia gama de frameworks que se adaptan a distintas necesidades. Su sintaxis clara y su ecosistema en constante evolución han impulsado su adopción en aplicaciones de todos los tamaños, desde sitios web sencillos hasta plataformas empresariales y sistemas de alto rendimiento.

Entre los frameworks más destacados, encontramos:

- Django, ideal para aplicaciones completas con una arquitectura robusta y herramientas integradas.
- Flask, una opción ligera y flexible para proyectos que requieren mayor personalización.
- FastAPI, optimizado para la creación de APIs modernas y de alto rendimiento, con soporte nativo para tipado estático y asincronismo.

A continuación, exploramos **Flask**, una excelente opción para quienes buscan simplicidad y flexibilidad en el desarrollo web.

Flask: Un Microframework Ligero y Flexible

Flask es un microframework minimalista para desarrollo web en Python, diseñado para proporcionar lo esencial sin imponer una estructura rígida. A diferencia de Django, que incluye una arquitectura completa con herramientas integradas, Flask permite a los desarrolladores elegir sus propias bibliotecas y estructuras, lo que lo convierte en una opción ideal para aplicaciones ligeras o que requieren mayor control sobre los componentes utilizados.

Características clave de Flask:

- Ligero y modular, lo que facilita la creación de aplicaciones escalables sin sobrecarga innecesaria.
- Uso de Jinja2, un motor de plantillas potente para generar interfaces web dinámicas.
- Extensibilidad, con compatibilidad para múltiples bibliotecas y plugins como Flask-SQLAlchemy para bases de datos y Flask-Login para autenticación.
- Soporte para APIs RESTful, facilitando el desarrollo de servicios web eficientes.
- Integración con WSGI y compatibilidad con servidores como Gunicorn y uWSGI para despliegue en producción.

Flask es ideal para desarrolladores que buscan control total sobre la estructura de su aplicación, permitiendo una mayor personalización sin la necesidad de seguir un patrón predefinido. Sin embargo, para proyectos más grandes y estructurados, Django puede ser una mejor alternativa, mientras que para APIs modernas y de alto rendimiento, FastAPI ofrece ventajas significativas.

Ejemplo básico de una aplicación web con Flask:

```
python
# Importa la clase Flask para crear la aplicación web
from flask import Flask

# Crea una instancia de la aplicación Flask
app = Flask(__name__)

# Define la ruta principal (la raíz del sitio web)
@app.route('/')
def home():
    # Devuelve un mensaje cuando se accede a la ruta principal
    return "¡Hola, Flask!"

# Verifica si el archivo se está ejecutando directamente
if __name__ == '__main__':
    # Inicia el servidor en modo depuración
    app.run(debug=True)
```

Este código crea un servidor web local en el puerto 5000 por defecto, y cuando abres <http://127.0.0.1:5000/> en tu navegador, verás ¡Hola, Flask!.

3.2. Visualización de datos en Dash

Dash es un framework basado en **Flask** que permite construir aplicaciones web interactivas en Python. La visualización de datos es un aspecto clave en el desarrollo de aplicaciones interactivas, ya que permite interpretar y comunicar información de manera efectiva. En el ecosistema de Python, existen varias librerías especializadas en la generación de gráficos, cada una con características únicas que determinan su idoneidad para distintos casos de uso.

Comparación general de las principales librerías

Librería	Características clave	Interactividad	Cuándo usarla	Página oficial
Matplotlib	Ofrece total flexibilidad en la creación de gráficos, incluyendo visualizaciones personalizadas y en 3D.	❌ No permite gráficos interactivos	Ideal para gráficos detallados y configuraciones altamente personalizadas.	matplotlib.org
Seaborn	Se especializa en gráficos estadísticos con temas estilizados y soporte para DataFrames de Pandas.	❌ No permite gráficos interactivos	Útil cuando se requiere analizar relaciones estadísticas en los datos.	seaborn.pydata.org
Plotly	Permite crear gráficos interactivos y optimizados para la web sin necesidad de configuraciones adicionales.	✅ Sí permite gráficos interactivos	Recomendado para dashboards dinámicos e interfaces web interactivas.	plotly.com

Matplotlib es una librería que ofrece un control total sobre la creación de gráficos, permitiendo visualizaciones altamente personalizadas y detalladas. Por otro lado, **Seaborn** se basa en Matplotlib y simplifica la generación de gráficos estadísticos avanzados con una sintaxis más intuitiva y estilos predeterminados atractivos.

Sabías que ...

Aunque Matplotlib y Seaborn generan gráficos estáticos, aún pueden integrarse en Dash usando FigureCanvasAgg. Esto permite renderizarlos como imágenes dentro de la aplicación. Sin embargo, si buscas interactividad, Plotly es la mejor opción para dashboards dinámicos en la web debido a su integración nativa con Dash.



3.2.1. Plotly en Dash (la opción nativa y recomendada)

Plotly es una de las librerías más potentes para la visualización de datos en Python, destacando por su capacidad de generar gráficos interactivos y optimizados para la web. Su integración con Dash es nativa, lo que permite crear dashboards dinámicos sin necesidad de configuraciones adicionales.

Dentro de Plotly, existen dos enfoques principales para la creación de gráficos: **Plotly Express** y **Plotly Graph Objects**. A continuación, exploraremos ambos enfoques y su implementación dentro de Dash.

Plotly Express

Es una biblioteca de Python que permite crear gráficos interactivos de manera rápida y sencilla. Es una interfaz de alto nivel de Plotly que simplifica la generación de visualizaciones sin necesidad de escribir código extenso.

Plotly Express incluye una serie de conjuntos de datos gratuitos integrados, diseñados para facilitar la creación y prueba de visualizaciones sin necesidad de cargar archivos externos. Estos datasets abarcan diversas áreas como biología, economía, geografía y más. A continuación, se muestran algunos de los conjuntos de datos gratuitos integrados con esta librería.

Conjunto de datos gratuitos integrados en Plotly Express:

Dataset	Descripción
<code>px.data.gapminder()</code>	Datos de países (PIB, esperanza de vida y población desde 1952 a 2007).
<code>px.data.iris()</code>	Datos de flores de Iris con medidas de pétalos y sépalos.
<code>px.data.tips()</code>	Datos de propinas en un restaurante, con información sobre el día, género, tamaño del grupo, etc.
<code>px.data.carshare()</code>	Ubicaciones de puntos de viaje compartido en una ciudad.
<code>px.data.election()</code>	Resultados de elecciones en distritos de Montreal (Canadá).
<code>px.data.wind()</code>	Datos sobre velocidades y direcciones del viento en diferentes ubicaciones.
<code>px.data.medals_long()</code>	Datos de medallas olímpicas en un formato largo.
<code>px.data.gapminder()</code>	Estadísticas globales de salud y economía por país y año.
<code>px.data.stocks()</code>	Precios de acciones de varias empresas de tecnología.

Sabías que ...

Plotly Express incluye varios conjuntos de datos gratuitos listos para usar, sin necesidad de descargar archivos externos. Puedes explorar datos de flores (Iris), tendencias económicas (Gapminder), propinas en restaurantes (Tips), entre otros. Si usas Plotly (sin Express), debes cargar los datos manualmente desde un archivo (CSV, Excel, JSON, etc.).



Ejemplo básico de gráfico de dispersión con Plotly Express:

```
python

import dash
from dash import dcc, html
import plotly.express as px

# Aplicación Dash
app = dash.Dash(__name__)

# Cargar la base de datos Iris
df = px.data.iris()

# Crear un gráfico de dispersión con Plotly Express
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species",
                 title="Relación entre ancho y largo del sépalo")

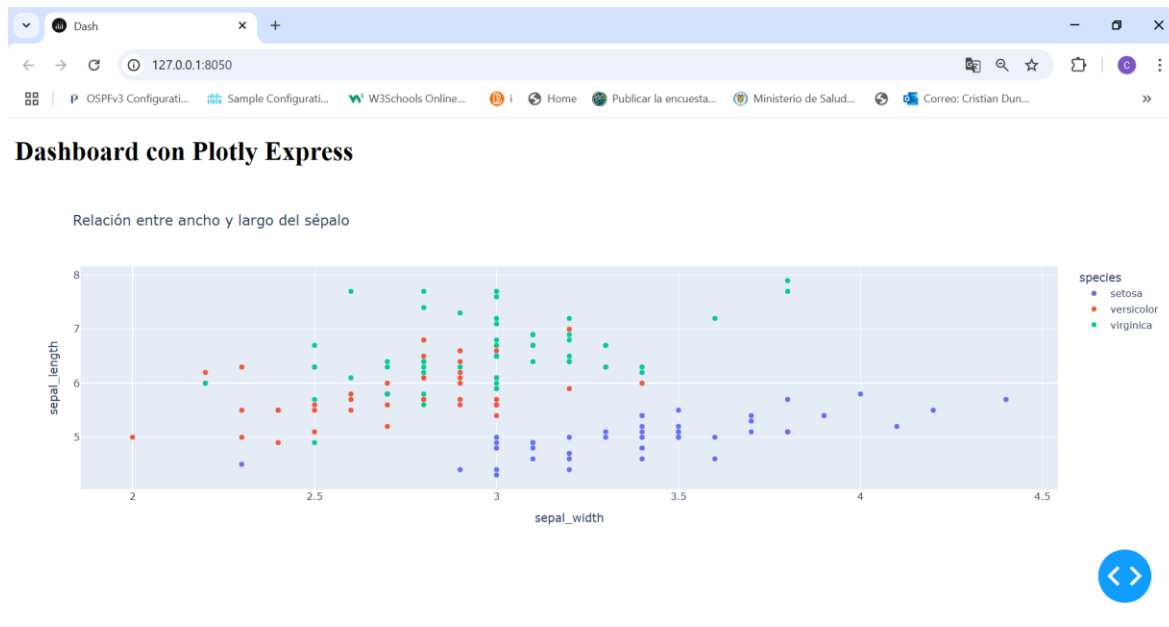
# Layout de la aplicación
app.layout = html.Div([
    html.H1("Dashboard con Graph Objects"),
    dcc.Graph(figure=fig) # Renderizar gráfico en Dash
])

if __name__ == '__main__':
    app.run_server(debug=True)
```

Este código ejecuta un servidor web local donde se despliega una aplicación web

interactiva con Dash y Plotly Express, visualizando un gráfico de dispersión basado en la base de datos Iris.

Ejemplo de ejecución



Plotly Graph Objects

Plotly Graph Objects es un enfoque más detallado y flexible para la creación de gráficos en Plotly. A diferencia de **Plotly Express**, que ofrece funciones simplificadas para generar visualizaciones rápidamente, **Graph Objects** permite un mayor control sobre cada aspecto del gráfico, como la personalización de trazas, ejes, leyendas y estilos.

Este enfoque es ideal para situaciones donde se requiere una configuración avanzada o gráficos altamente personalizados. Aunque su sintaxis es más extensa, proporciona una gran versatilidad para diseñar visualizaciones interactivas y adaptadas a necesidades específicas.

Nota: A diferencia de Plotly Express, Plotly Graph Objects no proporciona datasets integrados, por lo que es necesario cargar los datos desde fuentes externas o utilizar los conjuntos disponibles en Plotly Express.

Ejemplo básico de gráfico de dispersión con Graph Objects:

```
python

import dash
from dash import dcc, html
import plotly.graph_objects as go
import plotly.express as px

# Aplicación Dash
app = dash.Dash(__name__)

# Cargar la base de datos Iris
df = px.data.iris()

# Crear un gráfico con Graph Objects
fig = go.Figure()

# Agregar trazas manualmente para cada especie
for species in df["species"].unique():
    datos = df[df["species"] == species]
    fig.add_trace(go.Scatter(
        x=datos["sepal_width"], y=datos["sepal_length"],
        mode='markers', name=species
    ))

fig.update_layout(title="Gráfico de dispersión con Graph Objects")

# Layout de la aplicación
app.layout = html.Div([
    html.H1("Dashboard con Graph Objects"),
    dcc.Graph(figure=fig) # Renderizar gráfico en Dash
])

if __name__ == '__main__':
    app.run_server(debug=True)
```

Este código ejecuta un servidor web local donde se despliega una aplicación web interactiva con Dash y Plotly Graph Objects, visualizando un gráfico de dispersión basado en la base de datos Iris.

Comparación rápida:

Enfoque	Simplicidad	Personalización	Casos de uso
Plotly Express	✓ Fácil y rápido	♦ Limitado	Dashboards simples, prototipos rápidos
Graph Objects	♦ Más detallado	✓ Mayor control	Dashboards avanzados, gráficos complejos

3.3. Desarrollo de aplicaciones web analíticas con Python y Dash

Los dashboards interactivos permiten visualizar y analizar datos de manera dinámica, facilitando la toma de decisiones basada en información en tiempo real. Con **Dash** y **Plotly**, es posible desarrollar aplicaciones web que integran gráficos, filtros y otros componentes interactivos sin necesidad de conocimientos avanzados en desarrollo web.

A continuación, se detallan los aspectos clave para la creación de dashboards interactivos en Python con Dash:

Componentes clave de Dash:

- `dash.Dash`: Clase principal para inicializar la aplicación.
- `dash.html` (HTML Components): Permite estructurar la interfaz con etiquetas HTML (divs, párrafos, botones, etc.).
- `dash.dcc` (Core Components): Incluye elementos interactivos como gráficos, dropdowns, sliders, checkboxes, etc.
- `dash.dependencies`: Permite establecer interactividad con Input y Output.

Estructura recomendada para una aplicación web con Dash:

```
mi_dashboard/      # Carpeta raíz del proyecto
├── src/            # Carpeta que contiene el código fuente
│   ├── app.py      # Punto de entrada de la aplicación
│   ├── layout.py   # Definición del diseño y componentes
│   ├── callbacks.py # Funciones para interactividad
│   └── components/ # Carpeta con componentes personalizados de la interfaz
├── assets/         # Carpeta con archivos CSS, imágenes y otros recursos
├── data/           # Carpeta con archivos CSV, bases de datos, etc.
├── requirements.txt # Lista de dependencias para la instalación
├── Procfile        # Archivo opcional para despliegue en Heroku
└── README.md       # Documentación del proyecto
```

Nota: Para despliegue en plataformas *PaaS* como **Render**, debe reemplazarse el archivo `Procfile` por `render.yaml`, el cual contiene la configuración específica del entorno.

Beneficios de esta estructura:

- **Modularidad:** Facilita la organización del código.
- **Mantenimiento:** Permite actualizar partes de la aplicación sin afectar el resto.
- **Escalabilidad:** Se puede extender fácilmente agregando nuevas visualizaciones o funcionalidades.

Ejemplo básico de una aplicación web con Dash y Plotly:

```
python
# Importar los módulos necesarios de Dash
from dash import Dash, html, dcc, callback, Output, Input
import plotly.express as px # Importar Plotly Express para crear gráficos interactivos

# Crear la aplicación Dash sin argumentos adicionales
app = Dash()

# Cargar el conjunto de datos integrado en Plotly Express (Gapminder)
df = px.data.gapminder()

# Definir el diseño de la aplicación (layout)
app.layout = html.Div([
    # Título principal del dashboard
    html.H1("Dashboard con Plotly Express y Dropdown", style={'textAlign': 'center'}),
    # Dropdown (lista desplegable) para seleccionar un país
    dcc.Dropdown(
        df.country.unique(), # Lista de países extraída del conjunto de datos
        'Colombia', # Valor predeterminado del dropdown
        id='dropdown-pais'
    ),
    # Gráfico interactivo que se actualizará según el país seleccionado
    dcc.Graph(id='grafico-lineal')
])

# Callback para actualizar el gráfico según el país seleccionado en el dropdown
@callback(
    Output('grafico-lineal', 'figure'), # Define que se actualizará la figura del gráfico
    Input('dropdown-pais', 'value') # Se toma el valor seleccionado en el dropdown
)
def actualizar_grafico(pais):
    # Filtrar el conjunto de datos para el país seleccionado
    df_filtrado = df[df['country'] == pais]

    # Crear un gráfico de línea con el PIB per cápita a lo largo de los años
    fig = px.line(df_filtrado, x='year', y='gdpPercap',
                  title=f'PIB per cápita de {pais}', # Título dinámico con el país seleccionado
                  labels={'year': 'Año', 'gdpPercap': 'PIB per cápita'}) # Etiquetas de los ejes

    return fig # Retornar el gráfico actualizado

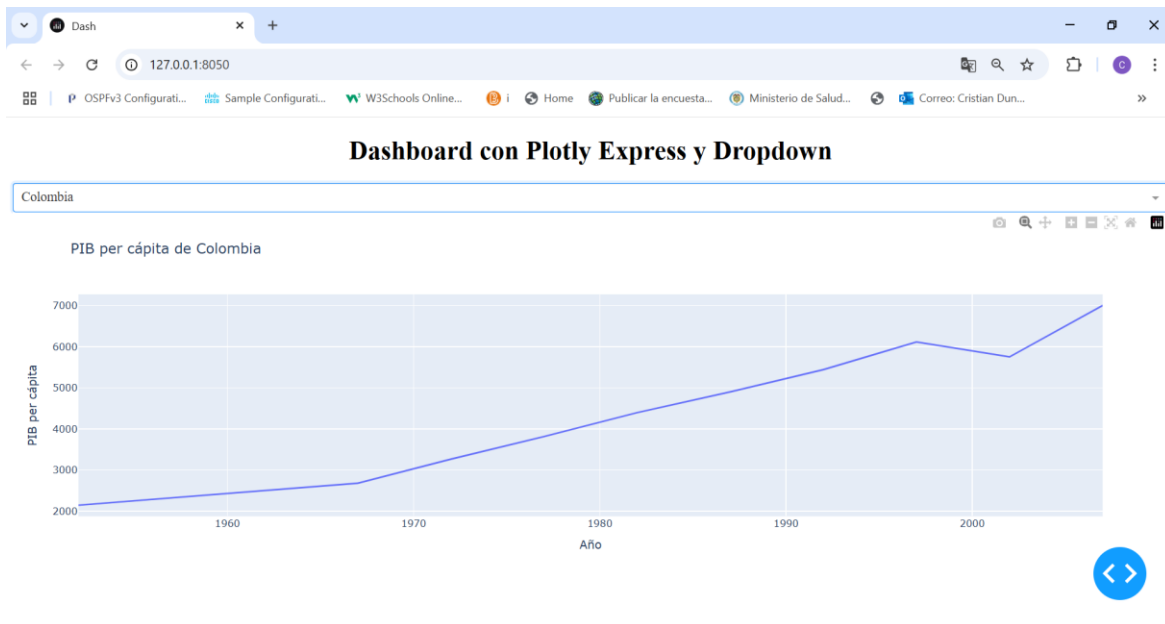
# Ejecutar la aplicación en el servidor local
if __name__ == '__main__':
    app.run_server(debug=True) # Modo debug activado para ver cambios en tiempo real
```

Este código permite visualizar de forma interactiva la evolución del PIB per cápita de distintos países usando **Dash** y **Plotly Express**. Los usuarios pueden elegir un país en la lista desplegable y ver cómo ha cambiado su PIB a lo largo del tiempo:

- Utiliza los conjuntos de datos de ejemplo de Gapminder (**px.data.gapminder**).
- Utiliza un gráfico de líneas interactivo (**dcc.Graph**) para representar la tendencia del PIB per cápita.

- Incluye una lista desplegable (**dcc Dropdown**) que permite seleccionar un país.
- Se emplea un **callback** para actualizar dinámicamente el gráfico en función de la selección del usuario, proporcionando una experiencia interactiva en tiempo real.
 - Entrada (Input('dropdown-pais', 'value')) → Captura el país seleccionado.
 - Salida (Output('grafico-lineal', 'figure')) → Actualiza el gráfico con datos filtrados.
- La función actualizar_grafico(pais) filtra los datos y devuelve un nuevo gráfico con la información del país seleccionado.

Ejemplo de ejecución:



Para que un dashboard sea realmente útil, debe permitir la interacción del usuario a través de elementos como listas desplegadas, botones, sliders, checkboxes e inputs de texto.

Principales componentes Dash:

Elemento	Componente	Descripción
Lista Desplegable	dcc.Dropdown	Permite al usuario seleccionar un país de una lista. Ideal para elegir un solo valor.
Slider	dcc.Slider	Permite al usuario seleccionar un rango de años. Útil para mostrar datos en un intervalo temporal específico.
Checkboxes	dcc.Checklist	Permite al usuario seleccionar múltiples países. Ideal para comparar varios países al mismo tiempo.
Input de Texto	dcc.Input	Permite al usuario escribir el nombre de un país y filtrar la lista de opciones.
Botones (RadioItems)	dcc.RadioItems	Permite al usuario elegir entre diferentes tipos de gráficos (ej. líneas, barras).
Callback	@app.callback	Actualiza el gráfico según la interacción del usuario con los elementos anteriores (dropdown, slider, etc.).

Callbacks:

Los callbacks en Dash son la base de la interactividad en una aplicación web. Funcionan como un mecanismo de reacción, donde un cambio en un componente (como la selección de un dropdown, el ingreso de texto o el clic en un botón) desencadena una actualización en otro componente (como un gráfico o una tabla).

Un callback se compone de tres elementos principales:

- 1 `@app.callback(Output, [Input])` → Define la relación entre la entrada y la salida.
- 2 Una función asociada → Se ejecuta automáticamente cada vez que cambia el Input.
- 3 Los argumentos de la función → Reciben los valores actualizados del Input y generan una respuesta.

Nota: Los callbacks son reactivos, se ejecutan automáticamente cuando cambian los valores en los Input.

Profundiza ...

Dash es un framework de Python desarrollado por Plotly para crear aplicaciones web interactivas de manera sencilla, sin necesidad de conocimientos avanzados en HTML, CSS o JavaScript. Si quieres aprender más, te invito a explorar los siguientes recursos:

- ♦ Página oficial de Dash → <https://dash.plotly.com/>
- ♦ Documentación de Dash → <https://dash.plotly.com/introduction>
- ♦ Curso introductorio en YouTube → <https://www.youtube.com/watch?v=hSPmj7mK6ng>



3.4. Manejo de datos con Dash

Dash es una excelente herramienta para construir aplicaciones web interactivas basadas en datos. Su verdadero potencial se maximiza cuando se integra con bases de datos, consume APIs en tiempo real y utiliza Pandas para la manipulación de datos. A continuación, exploramos cada uno de estos aspectos en detalle.

Integración con Bases de Datos (PostgreSQL)

Las aplicaciones Dash suelen manejar grandes volúmenes de datos, por lo que conectarlas con bases de datos relacionales como **SQLite** o **PostgreSQL** es una práctica común. PostgreSQL es más robusto y adecuado para aplicaciones escalables en la nube. Se puede integrar con SQLAlchemy y psycopg2.

Ejemplo básico de conexión en Dash con PostgreSQL:

```
python

from sqlalchemy import create_engine
import pandas as pd

# Conectar a PostgreSQL
DATABASE_URL = "postgresql://usuario:contraseña@localhost:5432/mi_basedatos"
engine = create_engine(DATABASE_URL)

# Leer datos
df = pd.read_sql("SELECT * FROM datos_paises", con=engine)
```

Este código se encarga de conectar una aplicación en Python a una base de datos PostgreSQL utilizando SQLAlchemy y Pandas para extraer datos de una tabla.

Consumo de APIs con Dash

Dash puede obtener datos en tiempo real desde APIs públicas o privadas mediante requests.

Ejemplo básico de conexión en Dash con APIs:

```
python

import requests
import pandas as pd

url = "https://covid-api.com/api/reports"
response = requests.get(url) # Hacer petición a la API
data = response.json() # Convertir a formato JSON

# Extraer datos relevantes y guardarlos en un DataFrame
df = pd.DataFrame(data["data"])[["region", "confirmed", "deaths", "recovered"]]

# Extraer el nombre del país dentro de la columna "region"
df["country"] = df["region"].apply(lambda x: x["name"])
df = df[["country", "confirmed", "deaths", "recovered"]]

print(df.head()) # Mostrar las primeras filas
```

El código anterior realiza una solicitud a la API <https://covid-api.com/api/reports> y extrae los datos de COVID-19 por país.

Procesamiento de datos con Pandas

A veces, los datos necesitan limpiarse antes de mostrarse en el dashboard. Pandas es una librería que nos ayuda a hacer esto.

Ejemplo básico para filtrar datos con Pandas

```
python

import pandas as pd

# Cargar el archivo CSV
df = pd.read_csv("data/gapminder.csv")

# Filtrar los datos de Colombia
df_colombia = df[df["country"] == "Colombia"]

print(df_colombia.head()) # Ver los primeros datos
```

Este código en Python utiliza la librería Pandas para leer y filtrar datos de un archivo CSV llamado gapminder. Luego, selecciona únicamente los registros correspondientes a Colombia y muestra los primeros 5 registros.

Profundiza ...

Pandas es una biblioteca de Python diseñada para la manipulación y análisis de datos de manera eficiente. Es ampliamente utilizada en ciencia de datos, ingeniería de datos y análisis estadístico gracias a su facilidad para manejar estructuras como tablas y series de datos.

- ◆ Página oficial de Pandas → <https://pandas.pydata.org/>
- ◆ Documentación de Pandas → <https://pandas.pydata.org/docs/>
- ◆ Introducción en YouTube → <https://www.youtube.com/watch?v=8ASjvOIyyl8&t=11s>



¿Qué método usar para el manejo de datos?

Método	¿Cuándo usarlo?	Ejemplo
SQLite	Para aplicaciones pequeñas con pocos datos.	Guardar registros de usuarios.
PostgreSQL	Para grandes volúmenes de datos en la nube.	Consultar información de millones de personas.
APIs	Para obtener datos en vivo de internet.	Mostrar clima, precios de criptomonedas.
Pandas	Para filtrar y transformar datos antes de mostrarlos.	Calcular promedios, filtrar países.

3.5. Autenticación y seguridad en Dash

Al desarrollar aplicaciones web con Dash, es crucial implementar mecanismos de autenticación y seguridad para proteger la información y restringir el acceso a usuarios autorizados. A continuación, se explican los aspectos clave en este proceso teniendo en cuenta que Dash no incluye autenticación integrada, pero se puede implementar con frameworks como **Flask-Login**, **Flask-JWT-Extended**, o usando autenticación basada en OAuth.

Métodos comunes de autenticación:

- Autenticación basada en formularios: Se solicita un usuario y contraseña antes de acceder a la aplicación.
- Autenticación con Single Sign-On (SSO): Se integra con servicios externos como Google, Azure AD o LDAP.
- Autenticación con API Key: Se requiere una clave secreta para acceder a la API de la aplicación.

Ejemplo básico de autenticación den Dash con Flask-Login:

```
python

from flask import Flask # Importamos Flask para crear la aplicación web
from flask_login import LoginManager, UserMixin, login_user, login_required

app = Flask(__name__) # Creamos la aplicación Flask
app.secret_key = "mi_secreto" # Clave secreta para gestionar sesiones

# Inicializamos el administrador de autenticación
login_manager = LoginManager(app)

# Clase de usuario basada en UserMixin para manejar sesiones
class Usuario(UserMixin):
    pass # Aquí se podría definir la lógica del usuario (por ahora se deja vacío)

# Función para cargar el usuario autenticado en la sesión
@login_manager.user_loader
def load_user(user_id):
    return Usuario() # Devuelve un usuario (debe mejorarse para casos reales)

# Ruta de inicio de sesión
@app.route('/login')
def login():
    usuario = Usuario() # Creamos una instancia de usuario
    login_user(usuario) # Iniciamos sesión con Flask-Login
    return "Usuario autenticado" # Mensaje de autenticación exitosa

# Ruta protegida que solo puede acceder un usuario autenticado
@app.route('/dashboard')
@login_required # Requiere autenticación para acceder
def dashboard():
    return "Bienvenido al Dashboard" # Mensaje de bienvenida tras autenticación

# Ejecutamos la aplicación si el script se ejecuta directamente
if __name__ == "__main__":
    app.run()
```

Este código crea una aplicación Flask con autenticación de usuarios mediante Flask-Login para gestionar sesiones, protegiendo la ruta /dashboard y permitiendo el acceso solo a usuarios autenticados, mientras que en /login un usuario se "autentica" sin validación real.

Nota: La ruta /dashboard usa el decorador @login_required, que restringe el acceso solo a usuarios autenticados. Si un usuario no ha iniciado sesión, se redirige automáticamente a la página de inicio de

sesión (por defecto /login).



¿Protección de Rutas y Control de Acceso?

No todos los usuarios deben tener acceso a todas las secciones de una aplicación Dash. Es importante definir roles y permisos para cada usuario.

◇ **Formas de proteger rutas:**

- Redirigir a un login si no está autenticado
- Mostrar contenido diferente según el usuario
- Restringir acciones sensibles

3.6. Despliegue de aplicaciones web dinámicas con Dash en la nube

El despliegue en la nube es uno de los procesos más importantes al llevar una aplicación web desde un entorno local a un entorno accesible por otros usuarios a través de internet. Para ello, existen varias plataformas y herramientas que facilitan este proceso. Una de las formas más comunes de desplegar aplicaciones en la nube es a través de **plataformas como servicio**, o **PaaS** (Platform as a Service). Estas plataformas permiten a los desarrolladores centrarse en el código y la funcionalidad sin tener que gestionar la infraestructura.

Introducción a PaaS: Heroku, Render, Railway

- **Heroku:** Es una de las plataformas PaaS más populares. Proporciona un entorno simplificado para desplegar aplicaciones en la nube. Con Heroku, solo necesitas subir el código fuente de tu aplicación y Heroku se encarga del resto: desde la infraestructura hasta la escalabilidad. Puedes utilizar Heroku para aplicaciones de diferentes lenguajes como Python, Ruby, Node.js, entre otros.
- **Render:** Es una plataforma similar a Heroku, pero con características más centradas en la automatización y escalabilidad. Permite el despliegue de aplicaciones web, APIs y bases de datos, todo desde una interfaz fácil de usar. Render tiene soporte para diversos lenguajes y permite realizar implementaciones de CI/CD (integración y entrega continua).
- **Railway:** Es otra opción para desplegar aplicaciones en la nube. Esta plataforma se enfoca en simplificar el proceso de despliegue mediante integración con servicios populares como bases de datos y herramientas de mensajería, y permite a los desarrolladores crear aplicaciones más rápidamente sin preocuparse por la infraestructura.



Modelos de Computación en la Nube

Los servicios en la nube se dividen en diferentes modelos según el nivel de control y gestión que ofrecen.:

IaaS (Infraestructura como Servicio): Proporciona servidores, almacenamiento y redes virtualizados bajo demanda. Ofrece flexibilidad y control, pero requiere conocimientos técnicos.

Ejemplos: AWS EC2, Google Compute Engine.

PaaS (Plataforma como Servicio): Brinda un entorno listo para desarrollar, probar y desplegar aplicaciones sin gestionar infraestructura. Facilita el desarrollo, pero limita el control. *Ejemplos:* Heroku, Google App Engine, Render.

SaaS (Software como Servicio): Aplicaciones listas para usar en la nube sin instalación local. Es accesible y fácil de usar, pero ofrece menos personalización. *Ejemplos:* Google Drive, Zoom, Office 365.

Despliegue de aplicaciones Flask y Dash en Render

Flask y Dash son frameworks ampliamente utilizados en Python para el desarrollo de aplicaciones web:

- **Flask** es ideal para aplicaciones ligeras y personalizables.
- **Dash**, basado en Flask, está orientado a la creación de dashboards interactivos, facilitando la visualización de datos.

Para desplegar estas aplicaciones en **Render**, se recomienda seguir la siguiente estructura y archivos clave:

- **src/**: Carpeta que contiene el código fuente de la aplicación.
 - **app.py**: Archivo principal donde se instancia la aplicación Dash. Dentro de este archivo, la instrucción **server = app.server** expone el servidor Flask subyacente, requisito indispensable para que Render pueda ejecutar la aplicación correctamente.
- **render.yaml**: Archivo de configuración específico para Render. Define aspectos como:
 - El entorno de ejecución (por ejemplo, Python 3.11),
 - Comando para iniciar la aplicación (como `gunicorn src.app:server`),
 - Variables de entorno necesarias para el entorno de producción.
- **requirements.txt**: Archivo que lista todas las dependencias del proyecto, incluyendo Dash, Flask y cualquier otra librería utilizada. Render lo usa para instalar automáticamente los paquetes requeridos.
- **README.md**: Documento descriptivo que sirve como guía técnica del proyecto. Incluye:
 - Resumen del propósito de la aplicación,
 - Instrucciones para ejecución local,
 - Detalles sobre el despliegue en Render.

Pasos desplegar aplicaciones Dash en Render:

❶ Crear un proyecto Dash:

- Empieza creando tu aplicación Dash en tu máquina local y asegúrate de que tenga una estructura adecuada. Crea un archivo principal llamado `app.py`, donde se define tu aplicación Dash como se muestra a continuación

```
python

import dash
from dash import html

app = dash.Dash(__name__)

app.layout = html.Div("¡Hola, Render! Esta es una app Dash desplegada.")

server = app.server # Necesario para Render

if __name__ == "__main__":
    app.run_server(debug=True)
```

❷ Crea el archivo `requirements.txt`

- Asegúrate de tener un archivo `requirements.txt` en tu proyecto. Este archivo debe incluir todas las dependencias necesarias para que Render instale automáticamente los paquetes al desplegar la aplicación (por ejemplo, Dash y otras librerías necesarias).

```
dash==2.0.0
unicorn==20.1.0
```

Nota: unicorn es obligatorio para ejecutar la aplicación en un servidor de producción.

❸ Subir el Proyecto a GitHub

Render se integra directamente con plataformas como **GitHub** o **GitLab**, por lo que es necesario tener el código fuente alojado en un repositorio, ya sea público o privado.

Pasos para subir el proyecto a GitHub:

1. Crea un nuevo repositorio en tu cuenta de GitHub
2. Desde tu terminal, inicializa el repositorio local y sube los archivos:

```
git init
git add .
git commit -m "Primera versión de la app Dash"
git branch -M main
git remote add origin https://github.com/usuario/repo.git
git push -u origin main
```

Nota: Si no deseas usar la terminal, también puedes crear directamente la estructura del proyecto desde la interfaz web de GitHub, agregando carpetas y archivos manualmente (como el `README.md`, `requirements.txt`, etc.), y luego conectarlo con Render para el despliegue.

4 Crear un nuevo Servicio en Render

1. Ve a [Render](#) y crea una cuenta si no tienes una.
2. En el dashboard, haz clic en New → Web Service.
3. Conéctalo con el repositorio de GitHub donde subiste tu código.
4. Configura los siguientes parámetros:
 - Runtime: Python 3.x
 - Start Command: gunicorn app:server
 - Environment: Auto-detect

5 Desplegar y Probar la Aplicación

Una vez tengas tu aplicación desarrollada y alojada en el repositorio de GitHub, puedes desplegarla en Render mediante tres métodos diferentes, según tu nivel de experiencia y preferencias:

Opción 1: Despliegue con render.yaml (Blueprint):

Ideal si ya cuentas con un archivo render.yaml y deseas un proceso automatizado y reproducible.

1. Asegúrate de tener el archivo render.yaml en la raíz del repositorio.
2. Asegúrate de que tu archivo render.yaml incluya la línea: plan: free, para que Render utilice automáticamente el plan gratuito.
3. Ve a <https://render.com> y selecciona New → Blueprint.
4. Conecta tu repositorio y Render detectará automáticamente el archivo render.yaml
5. Completa los campos del formulario:
 - Blueprint Name: Ingresa un nombre para identificar el Blueprint (por ejemplo: proyecto-dash-blueprint).
 - Branch: Verifica que esté seleccionada la rama correcta (por defecto, main).
 - Configuración del servicio: Asegúrate de que esté marcada la opción: Create all as new services
6. Haz clic en Deploy Blueprint.
7. Una vez desplegado, accede a la URL generada, por ejemplo:
<https://proyecto-dash.onrender.com>

Opción 2: Despliegue Manual desde la Web de Render

Ideal si no tienes un archivo render.yaml y prefieres configurar el servicio desde la interfaz web.

1. Importante: asegúrate de que tu archivo src/app.py tenga esta línea: server = app.server
2. Ve a <https://render.com> y selecciona New → Web Service.
3. Conecta tu cuenta de GitHub y selecciona el repositorio que contiene tu aplicación Dash.
4. Completa los campos del formulario:
 - Name: Escribe un nombre para tu servicio (por ejemplo: proyecto-dash).
 - Root Directory: Déjalo vacío si el archivo app.py está dentro de la carpeta src/.
 - Environment: Selecciona Python 3.

- Build Command: `pip install -r requirements.txt`
 - Start Command: `gunicorn src.app:server`
5. Selecciona el tipo de instancia: Marca la opción Free (gratuita), que incluye 512 Mb (RAM) y 1 CPU al mes (suficiente para pruebas o proyectos educativos).
 6. (Opcional) Si necesitas variables de entorno (por ejemplo, tokens o claves), agrégalas en la sección Environment Variables.
 7. Haz clic en "Create Web Service" o "Deploy Web Service" al final del formulario.
 8. Espera unos minutos mientras Render instala las dependencias y despliega la app.
 9. Una vez finalizado, Render mostrará una URL pública como:
`https://proyecto.onrender.com`

Opción 3: Despliegue con DashTools GUI (CLI + Interfaz Gráfica)

Ideal si quieres desplegar tu aplicación desde tu entorno local sin escribir archivos manualmente, aprovechando una interfaz gráfica asistida para publicar en Render.

La herramienta dash-tools permite lanzar una GUI desde la terminal para preparar el entorno y enviar la app a Render de forma guiada:

1. Instala la herramienta Dash Tools: `pip install dash-tools`
2. Desde la terminal, lanza la interfaz gráfica: `dashtools gui`
3. Esto abrirá en tu navegador local una interfaz con el explorador de archivos del proyecto.
4. En la interfaz podrás:
 - Verificar la existencia de archivos requeridos como:
 - `src/app.py`
 - `requirements.txt`
 - `render.yaml`
 - Comprobar que tu app contenga: `server = app.server`
 - Asignar un nombre al servicio en Render.
 - Generar automáticamente los archivos si no existen (`render.yaml`, `requirements.txt`).
8. Una vez todos los requisitos estén en verde , haz clic en "Deploy to Render".
9. Se abrirá automáticamente la página de Render con la configuración precargada desde los archivos generados.
10. Una vez finalizado, Render mostrará una URL pública como:
`https://proyecto.onrender.com`



Uso de memoria en Render (plan gratuito)

Render, en su plan gratuito, ofrece una instancia con 512 MB de RAM y 0.1 CPU, lo cual puede resultar insuficiente si:

- La aplicación carga archivos grandes directamente (como CSVs sin procesar).
- Se realizan cálculos complejos o transformaciones de datos en tiempo real.

Recomendación:

Para evitar errores como "worker failed", "application crashed" o que la app se quede cargando indefinidamente:

- Preprocesa los datos previamente (por ejemplo: en un archivo CSV liviano, o en formato reducido como JSON, Parquet).
- Evita cargar datasets completos en memoria al iniciar la app.
- Usa muestras de datos para pruebas o filtros paginados cuando sea posible.

Profundiza ...

Render es una plataforma de hosting en la nube para desplegar apps, APIs y bases de datos fácilmente. Es una alternativa a Heroku, destacando por su simplicidad, escalabilidad y soporte para varios lenguajes. Ideal para desarrolladores que buscan despliegue rápido sin gestionar infraestructura

- ♦ Página oficial de Render → <https://render.com/>
- ♦ Guía de implementación → <https://github.com/cristian-duney/desplegar-dash-render>
- ♦ Video tutorial en YouTube → <https://www.youtube.com/watch?v=XWJBToV5yww>



Referencias

Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual* (3rd ed.). CreateSpace Independent Publishing Platform.

Python Software Foundation. (2020). *PEP 8 — Style guide for Python code*.
<https://pep8.org>

Amazon Web Services. (s.f.). *¿Qué es Python?* <https://aws.amazon.com/es/what-is/python/>

Render. (s.f.). *Render: Cloud hosting for developers*. <https://render.com/>

Plotly. (s.f.). *Plotly: Python graphing library*. <https://plotly.com/python/>

Plotly. (s.f.). *Dash: Analytical web apps for Python & R*. <https://dash.plotly.com/>

Google Cloud. (s.f.). *¿Qué es PaaS?* <https://cloud.google.com/learn/what-is-paas>

Microsoft Azure. (s.f.). *Plataforma como servicio (PaaS)*. <https://azure.microsoft.com/es-es/overview/what-is-paas/>